Audited by CRED SHIELDS

CredShields

Smart Contract Audit

March 20, 2026 • Public Audit Report

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Ryze Protocol between January 28, 2026, and March 3, 2026. A retest was performed on March 19, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli(Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Ryze Protocol

AICPA SOC | SCS Advocates OWASP OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

Table of Contents

1. Executive Summary ——————————- 7

1.1 Residual Risk Summary 8

1.2 Accepted Design Tradeoffs 9

1.3 Recommended Future Improvements 10

1.4 State of Security 12

2. The Methodology ——————————- 13

2.1 Preparation Phase 13

2.1.1 Scope 13

2.1.2 Documentation 14

2.1.3 Audit Goals 14

2.2 Retesting Phase 14

2.3 Vulnerability classification and severity 15

2.4 CredShields staff 16

3. Findings Summary ——————————- 18

3.1 Findings Overview 18

3.1.1 Vulnerability Summary 18

5. Bug Reports ——————————— 22

Bug ID #H001 [Fixed] 22

Unrestricted Access to SSPCore.handleSSPoolSwap( ) Enables Session State Manipulation 22

Bug ID #H002 [Fixed] 24

Sessionized WBF Uses Target Weight Instead of Oracle Weight and Ignores Trade Direction 24

Bug ID #H003 [Fixed] 26

Pool Owner Has Unrestricted Admin Control Over Protocol-Level Parameters 26

Bug ID #H004 [Fixed] 28

calcSingleAssetJoinWithSmartShield Always Uses assets[1] Oracle Weight as oracleWeightOut Regardless of Token Ordering 28

Bug ID #H005 [Fixed] 30

weightFees Uses Hardcoded 50% Threshold for Heavy Side Flip, Incorrect for Non-50/50 Pools 30

Bug ID #H006 [Fixed] 32

Locked Principal Due to Forced Reward Harvest on Unstake 32

Bug ID #M001 [Fixed] 34

Missing Intent Fee Collection in executeJoinIntents 34

Bug ID #M002 [Acknowledged] 35

Pyth Price Feed Fee Refund Missing Across MultiHopRouter and RyzeRouter 35

Bug ID #M003 [Fixed] 36

Unconditional CEX Price Lookup Reverts Swaps on Non-SmartShield Pools 36
Bug ID #M004 [Fixed] 37
Intent Swap Fee Differentiation Missing When SmartShield Is Enabled 37
Bug ID #M005 [Fixed] 39
swapExactOut Reverse Pass Uses Router as Slicing User Instead of Actual Trader 39
Bug ID #M006 [Fixed] 41
Fallback Feeder Price in RyzeRouter.getQuote Ignores inAmount Parameter 41
Bug ID #M007 [Fixed] 42
swapExactOut Slippage Check Uses Reverse-Pass Estimates Before Forward Execution 42
Bug ID #M008 [Fixed] 44
updateConfigBatch Validates Old Config Instead of New Config Before Storage 44
Bug ID #M009 [Fixed] 46
WBR Tokens Permanently Stuck in Router During swapExactOut Forward Execution 46
Bug ID #M010 [Fixed] 48
swapExactOut Refund Uses Reverse-Pass Estimate Instead of Actual Amount Consumed 48
Bug ID #M011 [Acknowledged] 50
Missing Minimum Weight Enforcement in Basic (Non SmartShield) Swap Path 50
Bug ID #M012 [Fixed] 52
calculateSessionizedFees Ignores AMM-Computed Slippage and WBF, Uses Simplified Formula Instead 52
Bug ID #M013 [Fixed] 54
_calculateSessionizedFees Uses Undecayed Notional for Previous Fee Baseline 54
Bug ID #M014 [Fixed] 56
Sessionized Slippage Is Computed But Never Applied to Adjust Swap Output 56
Bug ID #M015 [Fixed] 58
_executeSwapExactOut Checks Input Token Balance Instead of Output Token Balance 58
Bug ID #M016 [Fixed] 60
createPool Allows Arbitrary Oracle and WBR Contracts Without Whitelisting 60
Bug ID #M017 [Fixed] 62
Fee Suppression Restores Swap Fee on Improving Trades That Should Pay Zero 62
Bug ID #M018 [Fixed] 64
Incorrect Slippage Fee Calculation in Basic Swap Path (quoteOutGivenInBasic and quoteInGivenOutBasic) 64
Bug ID #M019 [Fixed] 66
Perpetual Emergency Lockout via Cooldown Reset 66
Bug ID #M020 [Fixed] 67
Missing Solvency Invariant Allows Permanent Claim Failure 67
Bug ID #M021 [Fixed] 69
Permanent Reward Lock on Token Removal 69
Bug ID #M022 [Fixed] 71

Unbounded Reward Token List May Cause Gas-Based DoS 71
Bug ID #M023 [Fixed] 73
Reward Debt Desynchronization After Token Removal and Re-Whitelisting 73
Bug ID #M024 [Fixed] 75
removeActivity Reverts on Zero-Weight LP Pools While removeActivityFromTokens Silently Succeeds 75
Bug ID #M025 [Fixed] 77
Oracle Price Drift Causes Phantom Activity and Unfair Emission Allocation 77
Bug ID #M026 [Fixed] 79
Inverted Slippage Comparison in InGivenOutMath Disables Slippage for ExactOut SmartShield Swaps 79
Bug ID #L001 [Fixed] 80
totalFees Always Returns Zero in MultiHopRouter::_executeSwap() 80
Bug ID #L002 [Acknowledged] 82
Sequential Nonce Scheme Permanently Blocks Pending Intents When Earlier Nonce Becomes Unexecutable 82
Bug ID #L003 [Acknowledged] 84
setSessionConfig Allows emaAlpha == WAD Which Disables Trade Notional Decay Entirely 84
Bug ID #L004 [Fixed] 86
No Batch Size Limit on Intent Execution Functions 86
Bug ID #L005 [Fixed] 87
Fee Rounding Loss From Integer Division by Number of Hops 87
Bug ID #L006 [Partially Fixed] 88
Direct Token Donation Can Push Oracle Weights Below MIN_WEIGHT_WAD Causing CapReached Revert 88
Bug ID #L007 [Fixed] 90
Capped WBF Not Reflected in sessionResult Causing Inflated Session State 90
Bug ID #L008 [Not Fixed] 92
weightFees Charges WBF on Neutral Trades Where diff == 0 92
Bug ID #L009 [Acknowledged] 94
WBR Distribution Silently Capped When Treasury Has Insufficient Funds 94
Bug ID #L010 [Acknowledged] 96
Trade Slicing Protection Bypassed When SmartShield Is Disabled Despite Independent Config 96
Bug ID #L011 [Acknowledged] 98
Cumulative Value Regression Permanently Locks User Claims 98
Bug ID #L012 [Fixed] 99
Out-of-Order Epoch Submission Silently Accepts Unreachable Root 99
Bug ID #L013 [Fixed] 100
claim() Hardcodes currentEpoch Proof Race Condition 100

Bug ID #L014 [Acknowledged] 101
Role Event Bypass Causing Off-Chain Desync 101
Bug ID #L015 [Fixed] 102
Global Reward Accounting Undercount 102
Bug ID #L016 [Acknowledged] 104
Permissionless Reward Claim Enables Forced Harvest 104
Bug ID #L017 [Fixed] 105
Raw ecrecover Usage Without Signature Malleability Protection in verifyCexPrice 105
Bug ID #L018 [Fixed] 106
New Liquidity Bands Default to 0% Minimum Slippage Floor 106
Bug ID #L019 [Fixed] 108
Incorrect Weight Parameter in WBF Calculation Uses Math.min Instead of Math.max 108
Bug ID #L020 [Not Fixed] 110
Missing Handling of Fees on Transfer in ERC20 Token Transfers 110
Bug ID #L021 [Fixed] 111
Epoch Advancement Cap May Record Activity in Stale Epoch After Long Dormancy 111
Bug ID #L022 [Fixed] 112
Per-Wallet Emission Cap Excess Is Not Redistributed 112
Bug ID #L023 [Fixed] 113
Slippage Protection Bypass via Unbacked Weight Breaking Reward Payouts 113
Bug ID #L024 [Fixed] 115
setEpochLength Missing Same-Value Validation 115
Bug ID #L025 [Fixed] 116
Emission Threshold Creates Dead Zone for Low-Activity Categories 116
Bug ID #L026 [Fixed] 118
Potential Double Weight Application in removeActivity 118
Bug ID #L027 [Fixed] 119
totalStandardUnits Rounding Drift From Per-User Decay Settlements 119
Bug ID #L028 [Fixed] 120
Protocol Fees Incorrectly Calculated on Slippage-Adjusted Amounts in SmartShield Swap Paths 120
Bug ID #L029 [Fixed] 122
Missing Zero-Address Check in transferGovernance Can Permanently Lock Governance 122
Bug ID #L030 [Fixed] 123
Floating and Outdated Pragma 123
Bug ID #L031 [Partially Fixed] 125
Missing events in important functions 125
Bug ID #I001 [Fixed] 127
Incorrect Error Used for Zero Merkle Root 127
Bug ID #I002 [Fixed] 128

Redundant State Update 128
Bug ID #I003 [Fixed] 129
Missing zero address validations 129
Bug ID #I004 [Fixed] 130
Dead Code 130
Bug ID #I005 [Not Fixed] 131
powWadUp And sqrtWadUp Provide No Rounding-Up Guarantee For Fractional Exponents 131
Bug ID #I006 [Fixed] 133
Non-Standard ERC20 Approve Pattern in MultiHopRouter May Break For USDT-Style Tokens 133
Bug ID #I007 [Acknowledged] 135
Insufficient SSPCore Upgrade Timelock Delay 135
Bug ID #G001 [Not Fixed] 137
Cheaper conditional operators 137
Bug ID #G002 [Not Fixed] 139
Gas Optimization in Increments 139
6. The Disclosure ——————————- 141

1. Executive Summary ----------

Ryze Protocol engaged CredShields to perform a smart contract audit from January 28, 2026, to March 3, 2026. During this timeframe, 72 vulnerabilities were identified. **A retest was performed on March 19, 2026, and all High/Critical issues have been remediated and retested. Some low severity findings remain acknowledged or accepted.**

During the audit, 6 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Ryze Protocol" and should be prioritized for remediation.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Solidity Smart Contracts	0	6	26	31	7	2	**72**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Solidity Smart Contract's scope during the testing window while abiding by the policies set forth by Ryze Protocol's team.

1.1 Residual Risk Summary

This section provides a consolidated view of all findings that were not fully remediated as of the retest on March 19, 2026. Readers are encouraged to review these items alongside the executive summary to form a complete picture of the protocol's current security posture.

The remaining findings are primarily of low severity or relate to design decisions acknowledged by the Ryze Protocol team. These items do not represent immediate critical vulnerabilities but should be considered in the context of the protocol's operational and configuration assumptions.

Bug ID	Severity	Title	Status
M002	Medium	Pyth Price Feed Fee Refund Missing Across MultiHopRouter and RyzeRouter	Acknowledged
M011	Medium	Missing Minimum Weight Enforcement in Basic (Non SmartShield) Swap Path	Acknowledged
L002	Low	Sequential Nonce Scheme Permanently Blocks Pending Intents When Earlier Nonce Becomes Unexecutable	Acknowledged
L003	Low	setSessionConfig Allows emaAlpha == WAD Which Disables Trade Notional Decay Entirely	Acknowledged
L006	Low	Direct Token Donation Can Push Oracle Weights Below MIN_WEIGHT_WAD Causing CapReached Revert	Partially Fixed
L008	Low	weightFees Charges WBF on Neutral Trades Where diff == 0	Not Fixed
L009	Low	WBR Distribution Silently Capped When Treasury Has Insufficient Funds	Acknowledged
L010	Low	Trade Slicing Protection Bypassed When SmartShield Is Disabled Despite Independent Config	Acknowledged
L011	Low	Cumulative Value Regression Permanently Locks User Claims	Acknowledged
L014	Low	Role Event Bypass Causing Off-Chain Desync	Acknowledged
L016	Low	Permissionless Reward Claim Enables Forced Harvest	Acknowledged
L020	Low	Missing Handling of Fees on Transfer in ERC20 Token Transfers	Not Fixed
L031	Low	Missing Events in Important Functions	Partially Fixed
I005	Informational	powWadUp And sqrtWadUp Provide No Rounding-Up Guarantee For Fractional Exponents	Not Fixed
I007	Informational	Insufficient SSPCore Upgrade Timelock Delay	Acknowledged
G001	Gas	Cheaper Conditional Operators	Not Fixed

Bug ID	Severity	Title	Status
G002	Gas	Gas Optimization in Increments	Not Fixed

Status Legend: Acknowledged = team accepted the risk; Not Fixed = no remediation applied; Partially Fixed = partially addressed.

1.2 Accepted Design Tradeoffs

These items were reviewed by the Ryze Protocol team and accepted as intentional design decisions or acknowledged risks within the current system architecture.

1. **M002 — Pyth Price Feed Fee Refund Missing:** The team notes that users can estimate the exact Pyth update fee prior to submission. The absence of a refund mechanism is accepted, with the understanding that excess ETH sent due to fee fluctuations between estimation and execution will remain in the contract. The team has accepted this risk
2. **M011 — Missing Minimum Weight Enforcement in Basic Swap Path:** The Ryze team has confirmed that extreme weight checks are not required for basic (non-SmartShield) pools, as these pools operate under a different risk model
3. **L002 — Sequential Nonce Scheme:** The team has acknowledged that the sequential nonce design is intentional. Even with a bitmap-based approach, the practical resolution would require re-submission of the stuck nonce. This is considered expected behaviour.
4. **L003 — emaAlpha == WAD Disables Decay:** Acknowledged. Pool owners are responsible for correct configuration of session parameters. Setting emaAlpha to WAD is technically permissible and is treated as a known operator risk.
5. **L009 — WBR Distribution Silently Capped:** The team has confirmed this is expected behaviour and does not wish to implement a pending rewards recovery mechanism for underpaid WBR.
6. **L010 — Trade Slicing Bypassed When SmartShield Disabled:** Acknowledged. The coupling of trade slicing protection to SmartShield is an accepted architectural decision for the current release.
7. **L011 — Cumulative Value Regression Locks Claims:** Acknowledged. Off-chain validation is relied upon to prevent monotonic regression of cumulative reward values across epochs.
8. **L014 — Role Event Bypass via Direct AccessControl Calls:** Acknowledged. On-chain logic remains correct; the risk is limited to potential off-chain monitoring desync.

9. **L016 — Permissionless Reward Claim / Forced Harvest:** Acknowledged. Funds are transferred to the correct recipient; the griefing vector is accepted as low impact.
10. **I007 — Insufficient SSPCore Upgrade Timelock (2 minutes):** Acknowledged as a testnet-only configuration. The team has confirmed the timelock will be increased to 48 hours prior to mainnet deployment.

1.3 Recommended Future Improvements

These items were not remediated in the current audit cycle. While their immediate security impact is limited, addressing them prior to or shortly after mainnet launch is recommended.

1. **L006 — Token Donation DoS (Partially Fixed):** The root cause (use of `balanceOf` for oracle weight calculation) remains. Donation-triggered cap breaches are still possible. Full remediation would require switching to internal balance accounting. Economic disincentives (donated tokens are lost) and arbitrage incentives for self-correction partially mitigate the practical risk.
2. **L008 — WBF Charged on Neutral Trades (diff == 0):** The neutral-trade case where `diff == 0` continues to fall into the worsening branch and incur WBF. Adding an explicit neutral check would ensure users are not penalised for trades that have no net effect on pool weight balance.
3. **L020 — Missing Fee-on-Transfer Handling:** Tokens that charge a transfer fee may cause accounting inaccuracies. The team is encouraged to add pre/post balance checks or explicitly document that fee-on-transfer tokens are not supported.
4. **L031 — Missing Events (Partially Fixed):** Several functions still lack event emission. Complete coverage is recommended for audit trail integrity and off-chain monitoring reliability.
5. **I005 — powWadUp / sqrtWadUp Rounding Guarantee:** For fractional exponents, `powWadUp` and `sqrtWadUp` produce identical results to their rounding-down counterparts. Adding a ceiling adjustment after the PRBMath call would honour the rounding-up contract and protect the pool from marginal under-collection on non-equal-weight pools.

6. **G001 & G002 — Gas Optimisations:** Replacing `x > 0` with `x != 0` and `i++` with `++i` across the codebase represents straightforward gas savings. These are low-priority but advisable ahead of high-volume mainnet usage.

1.4 State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Ryze Protocol's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Ryze Protocol can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Ryze Protocol can future-proof its security posture and protect its assets.

2. The Methodology ----------

Ryze Protocol engaged CredShields to perform a Solidity Smart Contracts audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from January 28, 2026, to March 3, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon and the **deployed contracts conform to the GitHub commits mentioned below**:

IN SCOPE ASSETS
Audited Scope: • https://github.com/ryze-protocol/ryze-contracts/tree/be637dc73f702b292ce7cd4744ccaaa1c918de21 • https://github.com/ryze-protocol/ryze-contracts/tree/da219980eae9e7be375f7074bcd498383a6aacda **Retested Scope:** • https://github.com/ryze-protocol/ryze-contracts/tree/3da99b9ba3da792d41d840e6f15a3c03119dfb5b

This audit reflects the security posture of the Ryze Protocol smart contracts at the specified commit hashes in scope. Any modifications made after the completion of the audit and retest are outside the scope of this assessment.

2.1.2 Documentation

Documentation was provided by the Ryze team wherever required. .

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

Ryze Protocol is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 72 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Unrestricted Access to SSPCore.handleSSPoolSwap() Enables Session State Manipulation	**High**	Access Control (SCWE-016)	Fixed
Sessionized WBF Uses Target Weight Instead of Oracle Weight and Ignores Trade Direction	**High**	Business Logic (SC03-LogicErrors)	Fixed
Pool Owner Has Unrestricted Admin Control Over Protocol-Level Parameters	**High**	Access Control (SCWE-016)	Fixed
calcSingleAssetJoinWithSmartShield Always Uses assets[1] Oracle Weight as oracleWeightOut Regardless of Token Ordering	**High**	Business Logic (SC03-LogicErrors)	Fixed
weightFees Uses Hardcoded 50% Threshold for Heavy Side Flip, Incorrect for Non-50/50 Pools	**High**	Business Logic (SC03-LogicErrors)	Fixed
Locked Principal Due to Forced Reward Harvest on Unstake	**High**	Denial of Service (DoS)	Fixed
Missing Intent Fee Collection in executeJoinIntents	**Medium**	Inconsistent Accounting (SCWE-010)	Fixed
Pyth Price Feed Fee Refund Missing Across MultiHopRouter and RyzeRouter	**Medium**	Business Logic (SC03-LogicErrors)	Acknowledged
Unconditional CEX Price Lookup Reverts Swaps on Non-SmartShield Pools	**Medium**	Denial of Service (SCWE-087)	Fixed
Intent Swap Fee Differentiation Missing When SmartShield Is Enabled	**Medium**	Inconsistent Accounting (SCWE-010)	Fixed

swapExactOut Reverse Pass Uses Router as Slicing User Instead of Actual Trader	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
Fallback Feeder Price in RyzeRouter.getQuote Ignores inAmount Parameter	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
swapExactOut Slippage Check Uses Reverse-Pass Estimates Before Forward Execution	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
updateConfigBatch Validates Old Config Instead of New Config Before Storage	**Medium**	Lack of Input Validation (SC04-Lack Of Input Validation)	Fixed
WBR Tokens Permanently Stuck in Router During swapExactOut Forward Execution	**Medium**	Stuck Funds	Fixed
swapExactOut Refund Uses Reverse-Pass Estimate Instead of Actual Amount Consumed	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
Missing Minimum Weight Enforcement in Basic (Non SmartShield) Swap Path	**Medium**	Inconsistent Accounting (SCWE-010)	Acknowledged
calculateSessionizedFees Ignores AMM-Computed Slippage and WBF, Uses Simplified Formula Instead	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
_calculateSessionizedFees Uses Undecayed Notional for Previous Fee Baseline	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
Sessionized Slippage Is Computed But Never Applied to Adjust Swap Output	**Medium**	Dead Code	Fixed
_executeSwapExactOut Checks Input Token Balance Instead of Output Token Balance	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
createPool Allows Arbitrary Oracle and WBR Contracts Without Whitelisting	**Medium**	Access Control (SCWE-016)	Fixed
Fee Suppression Restores Swap Fee on Improving Trades That Should Pay Zero	**Medium**	Business Logic (SC03-LogicErrors)	Fixed
Incorrect Slippage Fee Calculation in Basic Swap Path (quoteOutGivenInBasic and quoteInGivenOutBasic)	**Medium**	Accounting / Incorrect Calculation	Fixed
Perpetual Emergency Lockout via Cooldown Reset	**Medium**	Governance DoS	Fixed
Missing Solvency Invariant Allows Permanent Claim Failure	**Medium**	Centralization / Accounting Risk	Fixed
Permanent Reward Lock on Token Removal	**Medium**	Governance-Controlled Funds Lock	Fixed
Unbounded Reward Token List May Cause Gas-Based DoS	**Medium**	Gas Denial of Service	Fixed
Reward Debt Desynchronization After Token Removal and Re-Whitelisting	**Medium**	State Management Flaw	Fixed
removeActivity Reverts on Zero-Weight LP Pools While removeActivityFromTokens Silently Succeeds	**Medium**	Inconsistent Validation	Fixed
Oracle Price Drift Causes Phantom Activity and Unfair Emission Allocation	**Medium**	Business Logic Flaw	Fixed
Inverted Slippage Comparison in InGivenOutMath Disables Slippage for ExactOut SmartShield Swaps	**Medium**	Logic Error	Fixed
totalFees Always Returns Zero in MultiHopRouter::_executeSwap()	**Low**	Inconsistent Accounting (SCWE-010)	Fixed

Sequential Nonce Scheme Permanently Blocks Pending Intents When Earlier Nonce Becomes Unexecutable	**Low**	Denial of Service (SCWE-087)	Acknowledged
setSessionConfig Allows emaAlpha == WAD Which Disables Trade Notional Decay Entirely	**Low**	Lack of Input Validation (SC04-Lack Of Input Validation)	Acknowledged
No Batch Size Limit on Intent Execution Functions	**Low**	Denial of Service (SCWE-087)	Fixed
Fee Rounding Loss From Integer Division by Number of Hops	**Low**	Rounding Error	Fixed
Direct Token Donation Can Push Oracle Weights Below MIN_WEIGHT_WAD Causing CapReached Revert	**Low**	Denial of Service	Partially Fixed
Capped WBF Not Reflected in sessionResult Causing Inflated Session State	**Low**	Business Logic (SC03-LogicErrors)	Fixed
weightFees Charges WBF on Neutral Trades Where diff == 0	**Low**	Business Logic (SC03-LogicErrors)	Not Fixed
WBR Distribution Silently Capped When Treasury Has Insufficient Funds	**Low**	Business Logic (SC03-LogicErrors)	Acknowledged
Trade Slicing Protection Bypassed When SmartShield Is Disabled Despite Independent Config	**Low**	Business Logic (SC03-LogicErrors)	Acknowledged
Cumulative Value Regression Permanently Locks User Claims	**Low**	State Integrity Risk	Acknowledged
Out-of-Order Epoch Submission Silently Accepts Unreachable Root	**Low**	Lack of Input Validation (SC04-Lack Of Input Validation)	Fixed
claim() Hardcodes currentEpoch Proof Race Condition	**Low**	Design Weakness	Fixed
Role Event Bypass Causing Off-Chain Desync	**Low**	Event Inconsistency	Acknowledged
Global Reward Accounting Undercount	**Low**	Accounting Inconsistency	Fixed
Permissionless Reward Claim Enables Forced Harvest	**Low**	Missing Access Control	Acknowledged
Raw ecrecover Usage Without Signature Malleability Protection in verifyCexPrice	**Low**	Cryptographic Weakness	Fixed
New Liquidity Bands Default to 0% Minimum Slippage Floor	**Low**	Logic Error	Fixed
Incorrect Weight Parameter in WBF Calculation Uses Math.min Instead of Math.max	**Low**	Logic Error	Fixed
Missing Handling of Fees on Transfer in ERC20 Token Transfers	**Low**	Calculation Inaccuracy	Not Fixed
Epoch Advancement Cap May Record Activity in Stale Epoch After Long Dormancy	**Low**	SeverityLow	Fixed
Per-Wallet Emission Cap Excess Is Not Redistributed	**Low**	Business Logic Flaw	Fixed
Slippage Protection Bypass via Unbacked Weight Breaking Reward Payouts	**Low**	Business Logic (SC03-LogicErrors)	Fixed
setEpochLength Missing Same-Value Validation	**Low**	SeverityLow	Fixed
Emission Threshold Creates Dead Zone for Low-Activity Categories	**Low**	Business Logic Flaw	Fixed
Potential Double Weight Application in removeActivity	**Low**	SeverityLow	Fixed

totalStandardUnits Rounding Drift From Per-User Decay Settlements	**Low**	SeverityLow	Fixed
Protocol Fees Incorrectly Calculated on Slippage-Adjusted Amounts in SmartShield Swap Paths	**Low**	Accounting Error	Fixed
Missing Zero-Address Check in transferGovernance Can Permanently Lock Governance	**Low**	Input Validation	Fixed
Floating and Outdated Pragma	**Low**	Floating Pragma (SCWE-060)	Fixed
Missing events in important functions	**Low**	Missing Best Practices	Partially Fixed
Incorrect Error Used for Zero Merkle Root	**Informational**	Incorrect Error	Fixed
Redundant State Update	**Informational**	SeverityInformational	Fixed
Missing zero address validations	**Informational**	Missing Input Validation (SC04-Lack Of Input Validation)	Fixed
Dead Code	**Informational**	Code With No Effects (SCWE-062)	Fixed
powWadUp And sqrtWadUp Provide No Rounding-Up Guarantee For Fractional Exponents	**Informational**	Informational	Not Fixed
Non-Standard ERC20 Approve Pattern in MultiHopRouter May Break For USDT-Style Tokens	**Informational**	Best Practice	Fixed
Insufficient SSPCore Upgrade Timelock Delay	**Informational**	Governance	Acknowledged
Cheaper conditional operators	Gas	Gas Optimization (SCWE-082)	Not Fixed
Gas Optimization in Increments	Gas	Gas optimization (SCWE-082)	Not Fixed

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #H001 [Fixed]

Unrestricted Access to SSPCore.handleSSPoolSwap() Enables Session State Manipulation

Vulnerability Type
Access Control (SCWE-016)

Severity
High

Description
The SSPCore.handleSSPoolSwap() function is declared as public with no access control modifier, no msg.sender validation, and no whitelist check. Any external account or contract can call this function directly. While the downstream TradeSlicingProtection contract enforces that only authorized SSPCore contracts may call updateSessionState() and calculateSessionizedFees(), this check validates that the caller is the SSPCore contract itself, not that the original invoker of SSPCore is a legitimate WeightedPool. Since handleSSPoolSwap executes on the SSPCore contract, all downstream calls to TradeSlicingProtection originate from the SSPCore address and pass the authorization check regardless of who triggered handleSSPoolSwap in the first place.

This means an attacker can call handleSSPoolSwap() directly with crafted parameters, supplying any user address, any pool address, arbitrary amountInOrOut, fabricated tokenPrices, and a tradeSlicingProtection reference pointing to the real TradeSlicingProtection contract. The function will compute a swap quote using the attacker controlled inputs, then call tradeSlicingProtection.updateSessionState() with the attacker chosen user address, modifying that user's trading session data. The access check inside updateSessionState passes because msg.sender is the authorized SSPCore contract.

Affected Code

- SSPCore.sol#L33

Impacts
An attacker can inflate a victim's cumulativeNotional in TradeSlicingProtection by calling handleSSPoolSwap with the victim's address as the user parameter and a large amountInOrOut.

This causes the victim's subsequent legitimate trades to be treated as part of a large cumulative session, resulting in significantly higher sessionized slippage and WBF fees than they should pay.

Conversely, an attacker can reset their own session by calling handleSSPoolSwap with parameters that trigger a session reset (for example, switching the trade direction), allowing them to circumvent cumulative fee escalation designed to prevent trade slicing. This effectively defeats the entire purpose of the trade slicing protection system.

Remediation

It is recommended to add caller authorization to SSPCore.handleSSPoolSwap() that restricts invocation to registered WeightedPool contracts.

Retest

Added authorizedPools mapping and pool registration system to SSPCore.handleSSPoolSwap(), restricting calls to registered pool contracts only.

Bug ID #H002 [Fixed]

Sessionized WBF Uses Target Weight Instead of Oracle Weight and Ignores Trade Direction

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
High

Description
_calculateTotalWbfForNotional computes sessionized WBF using currentWeight, which suffers from two compounding issues:
1. Target weight passed instead of actual oracle weight (primary)
The call sites in OutGivenInMath and InGivenOutMath pass inAsset.weight the configured target weight as currentWeight:

```
// OutGivenInMath / InGivenOutMath
sessionResult = tradeSlicingProtection.calculateSessionizedFees(
  ...,
  inAsset.weight,    // <<<< target weight, NOT oracle weight
  weightBreakingReward
);
```

But _calculateTotalWbfForNotional passes this to computeWBFCharge, which expects the actual post-trade oracle minority weight (equivalent to Math.min(finIn, finOut) from weightFees):

```
uint256 wbfRate = weightBreakingReward.computeWBFCharge(
  notional, UD60x18.wrap(currentWeight), ...
);
```

Concrete example (50/50 target pool, oracle weights 0.75/0.25):

Target: inAsset.weight = 0.5e18
Actual minority weight: Math.min(finIn, finOut) = 0.25e18
WBF with target weight (0.5): F_pow(0.5) = 5 bps, lambdaBlend(0.5) = 0 → low WBF
WBF with actual weight (0.25): F_pow(0.75) = 77 bps, lambdaBlend(0.75) = 0.5 → high WBF

The sessionized WBF is massively understated for imbalanced pools, making trade slicing protection ineffective against WBF splitting attacks.

2. Same weight applied to all session trades regardless of direction (compounding)
Both previous and current cumulative WBF use the same currentWeight:

```
uint256 totalWbfForCumulative =
  _calculateTotalWbfForNotional(newTotalNotional, currentWeight, ...);
uint256 totalWbfForPrevious =
  _calculateTotalWbfForNotional(session.cumulativeNotional, currentWeight, ...);
```

The session does not track whether individual trades were improving (WBR) or worsening (WBF). If a previous trade improved pool balance (WBR given, WBF = 0), the sessionization still applies WBF retroactively for that trade's notional. Conversely, if the current trade improves the pool, it still gets charged incremental WBF from the session accumulation.

Affected Code

- InGivenOutMath.sol#L230
- OutGivenInMath.sol#L232
- TradeSlicingProtection.sol#L479-L488

Impacts

The sessionized WBF is massively understated for imbalanced pools because the target weight (e.g., 0.5) is used instead of the actual oracle minority weight (e.g., 0.25), making computeWBFCharge produce far lower fees than intended. This directly defeats trade slicing protection; an attacker can split a large worsening trade into multiple smaller ones and pay significantly less cumulative WBF than a single-shot trade would incur. The issue is compounded by the lack of direction tracking, which causes improving trades within a session to be retroactively penalized with WBF.

Remediation

Pass the actual post-trade oracle minority weight from the swap math libraries and Track trade direction within sessions to avoid applying WBF to improving trades.

Retest

weightFees now returns oracleWeightHeavyWad (Math.max) instead of target weight for WBF calculation; session model rewritten with signed netFlow to track trade direction; WBF only charged when isWorseningTrade is true

Bug ID #H003 [Fixed]

Pool Owner Has Unrestricted Admin Control Over Protocol-Level Parameters

Vulnerability Type
Access Control (SCWE-016)

Severity
High

Description
In WeightedPool, the owner (set as admin during createPool) has unrestricted access to all administrative functions. Since createPool is permissionless, any user who creates a pool becomes its owner and gains control over critical protocol-level parameters:

Fee destination hijacking:
updateAddresses() change treasury, protocolAddress, distributionAddress to attacker-controlled addresses
updateFeeRatios() redirect 100% of swap/taker/slippage/WBF fees to any destination

Security infrastructure replacement:
updateWeightBreakingReward() replace WBR contract with a malicious one
updateTradeSlicingProtection() replace trade slicing with a no-op
proposeSSPCoreUpgrade() replace SSPCore
setTrustedSigner() add attacker as trusted CEX price signer
setAuthorizedRouter() authorize malicious routers

Pool parameter manipulation:
updateParams() change swap fees, taker fees, SmartShield toggle
updateFeesAndMinSlippage() modify fee structure and slippage bounds

Attack scenario:
Attacker calls createPool() with themselves as admin
Pool is registered as isPool = true in the factory
Attacker calls updateAddresses() to redirect all fees to their wallet
Attacker calls setTrustedSigner() to add themselves as CEX price signer
Users trade on the pool (it appears legitimate via factory registry)
All fees flow to the attacker; attacker can manipulate prices via trusted signer

Affected Code

- WeightedPool.sol#L855
- WeightedPool.sol#L868
- WeightedPool.sol#L903
- WeightedPool.sol#L908
- WeightedPool.sol#L914
- WeightedPool.sol#L898
- WeightedPool.sol#L893
- WeightedPool.sol#L773

Impacts

The pool owner has complete control over fee destinations, security infrastructure, and pool parameters. Combined with permissionless pool creation, any attacker can create a factory-registered pool and redirect all protocol fees to themselves, replace critical security contracts, and manipulate pricing resulting in direct fund loss for any user trading on the pool.

Remediation

Separate pool-level admin from protocol-level admin. Protocol-critical operations should require the factory owner or a dedicated protocol governance address.

Retest

createPool() now enforces factory owner as pool owner (removed admin parameter); added pool creator whitelist; factory uses Ownable2Step for two-step ownership transfer

Bug ID #H004 [Fixed]

calcSingleAssetJoinWithSmartShield Always Uses assets[1] Oracle Weight as oracleWeightOut Regardless of Token Ordering

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
High

Description
In SingleJoinMath.calcSingleAssetJoinWithSmartShield(), the oracle weight of the "other" token (oracleWeightOut) is computed by always discarding the first return value from oracleNormalizedWeightsWithDecimals(assets[0], assets[1], tokenPrices) and taking the second:

```
(, oracleWeightOut) =
  WeightedPoolMath.oracleNormalizedWeightsWithDecimals(assets[0], assets[1],
tokenPrices);
```

This means oracleWeightOut always equals the oracle weight of assets[1], regardless of which token is tokenIn. When tokenIn == assets[1], the variable holds tokenIn's own oracle weight instead of the other token's weight. This incorrect weightMultiplier is passed into calcJoinValueWithSlippage, distorting the slippage calculation and consequently the number of LP shares minted.
Example:

- Pool: WETH (assets[0], balance = 100, oracle weight = 0.4) and USDC (assets[1], balance = 300,000, oracle weight = 0.6)
- User joins with 10,000 USDC (tokenIn = assets[1]), TVL = $500K, totalSupply = 500K
- oracleWeightOut should be 0.4 (WETH's weight) but code returns 0.6 (USDC's own weight)
- Buggy (weight = 0.6): weightedAmount = 6,000 → slippage = $2,078 → sharesOut = 7,922
- Correct (weight = 0.4): weightedAmount = 4,000 → slippage = $1,368 → sharesOut = 8,632
- User receives ~8.2% fewer LP shares than they should

Affected Code

- SingleJoinMath.sol#L216-L218

Impacts

When tokenIn == assets[1], the oracleWeightOut incorrectly reflects tokenIn's own oracle weight instead of the other token's weight, causing the slippage calculation in calcJoinValueWithSlippage to be systematically distorted. If tokenIn is the heavier asset, users receive fewer LP shares than deserved (direct loss of funds). If tokenIn is the lighter asset, users receive excess LP shares, diluting existing LPs and enabling an attacker to mint inflated shares and then exit proportionally to extract value from the pool.

Remediation

Capture both return values from oracleNormalizedWeightsWithDecimals and select the correct one based on which token is tokenIn. If inPa.token matches assets[0].token, use the second return value (weight of assets[1], the other token); otherwise use the first return value (weight of assets[0], the other token). This ensures oracleWeightOut always reflects the other token's oracle weight regardless of token ordering.

Retest

SingleJoinMath.calculateSingleAssetJoinShares() now correctly passes inPa and otherToken to oracleNormalizedWeightsWithDecimals() instead of hardcoded assets[0], assets[1] ordering.

Bug ID #H005 [Fixed]

weightFees Uses Hardcoded 50% Threshold for Heavy Side Flip, Incorrect for Non-50/50 Pools

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
High

Description
In WeightedPoolMath.weightFees(), the heavy side flip detection uses a hardcoded 0.5e18 threshold to determine which asset is "heavy":

```
bool initHeavySide = initIn > 0.5e18;
bool finHeavySide = finIn > 0.5e18;
bool flipped = initHeavySide != finHeavySide;
```

This is only correct for 50/50 pools. For any non-50/50 pool (e.g., 70/30, 80/20), oracle weights can cross the 0.5 midpoint while still improving towards the configured target weights. However, the flipped check takes priority over the diff calculation, forcing the trade into the worsening branch.

Example (70/30 pool, target weights: tokenIn = 0.7, tokenOut = 0.3):

- Pre-trade oracle weights: initIn = 0.48, initOut = 0.52 (dist from target = 0.22)
- Post-trade oracle weights: finIn = 0.52, finOut = 0.48 (dist from target = 0.18)
- initHeavySide = false (0.48 < 0.5), finHeavySide = true (0.52 > 0.5) → flipped = true
- But diff = 0.18 - 0.22 = -0.04 → trade improved pool (moved closer to 0.7 target)
- Because flipped is checked first, the trade is treated as worsening → WBF charged

This produces three cascading failures for all non-50/50 pools:

- WBF charged on improving trades the flip check overrides the distance-based improvement detection, so users who help rebalance the pool are penalized with WBF
- WBR never given the diff < 0 branch (where WBR is computed) is unreachable when oracle weights cross 0.5, so users never receive rewards for fixing the pool
- Fee suppression never fires shouldApplyFeeSuppression is only set in the diff < 0 branch, so improving trades still pay full swap and taker fees

Note: For 50/50 pools, a flip across 0.5 is a genuine pendulum swing past the target, so treating it as worsening is debatable. But for non-50/50 pools, crossing 0.5 has no relation to the target it's an arbitrary boundary that incorrectly triggers the worsening path.

This produces three cascading failures for non-50/50 pools:

- WBF charged on improving trades the flip check overrides the distance-based improvement detection, so users who help rebalance the pool are penalized with WBF
- WBR never given the diff < 0 branch (where WBR is computed) is unreachable when oracle weights cross 0.5, so users never receive rewards for fixing the pool
- Fee suppression never fires shouldApplyFeeSuppression is only set in the diff < 0 branch, so improving trades still pay full swap and taker fees

Affected Code

- WeightedPoolMath.sol#L562C9-L565C1

Impacts

In any non-50/50 pool, the WBF/WBR incentive mechanism is inverted. Improving trades that move oracle weights across the 0.5 midpoint are penalized with WBF instead of rewarded with WBR, and fee suppression never activates. This discourages pool rebalancing, causing persistent weight imbalance and increased LVR (loss-versus-rebalancing) for LPs.

Remediation

Use the configured target weights instead of the hardcoded 0.5 midpoint for heavy side flip detection. The flip should be determined by whether the heavy side relative to the target weights has changed, not relative to 50%.

Retest

This issue is fixed.

Bug ID #H006 [Fixed]

Locked Principal Due to Forced Reward Harvest on Unstake

Vulnerability Type
Denial of Service (DoS)

Severity
High

Description
The unstake() function in LPGaugeVault.sol forces the execution of _harvestAndSyncRewardDebt() before returning LP tokens to the caller. During this harvest, the internal function iterates over the entire whitelistedRewardTokenList array and invokes _safeRewardTransfer() for each token that has a non-zero pending reward balance.

The _safeRewardTransfer() function uses OpenZeppelin's safeTransfer, which reverts on any transfer failure. If any single reward token transfer reverts for example, due to a paused token contract, a blacklisted recipient address, or non-standard ERC-20 behavior the entire unstake() transaction reverts. Because the principal withdrawal is tightly coupled with the reward harvest in a single atomic transaction, users cannot retrieve their staked LP tokens unless every reward token transfer succeeds without exception.

Affected Code

- LPGaugeVault.sol#L136-L159
- LPGaugeVault.sol#L185-L203

Impacts
Users may be permanently unable to withdraw their staked LP tokens if even one whitelisted reward token becomes non-transferable. This effectively locks user principal for an indefinite period until either governance removes the problematic reward token from the whitelist or the token contract resumes normal transfer operations. The risk scales with the number of whitelisted reward tokens each additional token introduces another potential point of failure that can block all unstaking activity across the vault.

Remediation
Decouple reward harvesting from unstake() and allow users to withdraw principal independently without executing any reward token transfers.

Retest

This issue has been fixed.

Bug ID #M001 [Fixed]

Missing Intent Fee Collection in executeJoinIntents

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
Medium

Description
The MultiHopRouter contract implements two intent-based execution paths: executeSwapIntents for gasless token swaps and executeJoinIntents for gasless liquidity provision. Both paths rely on whitelisted relayers who pay gas on behalf of users and are compensated through intent fees. In _executeSwapIntent, after the swap is executed, the function checks whether intentFee is greater than zero and, if so, transfers the configured intentFee amount of WETH from the user's wallet to the protocol's feeCollector address using safeTransferFrom. This fee compensates the relayer for gas costs and provides protocol revenue. However, in _executeJoinIntent, no such fee collection logic exists. The function pulls the user's deposit token, approves the pool, calls joinSingle to mint LP shares, and returns without ever checking intentFee, without transferring any WETH, and without returning a feeCollected value.

Affected Code

- MultiHopRouter.sol#L570C1-L615C6

Impacts
Relayers who execute join intents receive no fee compensation for gas expenditures, creating an economic disincentive to process join intents.

Remediation
It is suggested to add intent fee collection logic to _executeJoinIntent that mirrors the fee mechanism in _executeSwapIntent.

Retest
This issue is fixed by adding intent fees in all intent operation

Bug ID #M002 [Acknowledged]

Pyth Price Feed Fee Refund Missing Across MultiHopRouter and RyzeRouter

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
When a user calls swapExactIn or swapExactOut on MultiHopRouter, they send ETH via msg.value to pay for Pyth oracle price feed updates. The MultiHopRouter forwards the entire msg.value to RyzeRouter.updatePriceFeeds(). Inside RyzeRouter, the function calculates the exact fee required by calling IPyth(pythContract).getUpdateFee(updateData), validates that msg.value is sufficient, and then forwards only the exact updateFee amount to the Pyth contract. The difference between msg.value and the actual updateFee which could be substantial since users cannot predict the exact fee at transaction signing time remains trapped in the RyzeRouter contract. Neither RyzeRouter nor MultiHopRouter refunds the excess ETH to the caller, and neither contract has a receive() function, a withdraw() function, or any other mechanism to recover these funds.

Affected Code

- MultiHopRouter.sol#L218C9-L222C1
- RyzeRouter.sol#L241C4-L247C6

Impacts
Every Pyth-enabled swap or join intent where msg.value exceeds the actual Pyth update fee results in ETH permanently locked in the RyzeRouter

Remediation
It is recommended to implement refund logic for pyth price feed update fees.

Retest
Client acknowledged. The team states users can estimate the exact Pyth update fee before submitting the transaction, avoiding excess ETH. However, the refund mechanism is still absent in RyzeRouter.updatePriceFeeds() any excess ETH sent (e.g., due to fee changes between estimation and execution) remains locked in the contract with no recovery path. Risk accepted by the team.

Bug ID #M003 [Fixed]

Unconditional CEX Price Lookup Reverts Swaps on Non-SmartShield Pools

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Medium

Description
In MultiHopRouter::_executeSwap(), every hop unconditionally calls SwapOperations.findCexPricesCalldata(hop.tokenIn, hop.tokenOut, cexPriceData) to locate CEX price entries for the token pair before executing the swap on the pool. The findCexPricesCalldata function iterates over the provided cexPriceData array searching for matching token addresses. If either token is not found including when the array is empty the function reverts with InvalidCexPriceData() because the returned pairCexPrice elements retain their default address(0) token values, which triggers the final validation check.

Affected Code

- MultiHopRouter.sol#L413C13-L414C106

Impacts
All swap operations routed through pools with SmartShield disabled are forced to include unnecessary CEX price data for every token in the path. If the caller does not provide this data, or provides an empty array which is the logical behavior when SmartShield is not in use the transaction reverts before reaching the pool.

Remediation
It is suggested to add a conditional check before calling findCexPricesCalldata that queries the target pool's SmartShield status. If SmartShield is disabled for the current hop's pool, skip the CEX price lookup.

Retest
This issue has been fixed by applying recommended remediation

Bug ID #M004 [Fixed]

Intent Swap Fee Differentiation Missing When SmartShield Is Enabled

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
Medium

Description
The WeightedPool contract configures two separate fee schedules for swaps: standard fees (swapFeeWad, takerFeeWad) applied to direct user swaps, and intent fees (intentSwapFeeWad, intentTakerFeeWad) applied to gasless intent based swaps submitted through relayers. This dual fee structure exists because intent swaps carry additional relayer costs and may warrant different pricing. In _executeSwapExactIn, the isIntentSwap boolean parameter is passed through to distinguish between these two types.

When SmartShield is disabled, the function calls quoteOutGivenInBasic, which correctly uses the isIntentSwap flag to select the appropriate fee tier. Specifically, it checks isIntentSwap ? params.intentSwapFeeWad : params.swapFeeWad for the swap fee and isIntentSwap ? params.intentTakerFeeWad : params.takerFeeWad for the taker fee, ensuring that intent swaps are charged the intent specific rates.

However, when SmartShield is enabled, the function calls sspCore.handleSSPoolSwap(), which does not accept an isIntentSwap parameter at all. The isIntentSwap value passed into _executeSwapExactIn is silently dropped and never forwarded. Inside the SmartShield path, OutGivenInMath.quoteOutGivenInSS unconditionally uses params.swapFeeWad and params.takerFeeWad for all swaps, regardless of whether the swap originated from a direct user call or a signed intent. The intentSwapFeeWad and intentTakerFeeWad parameters stored in the pool's configuration are never consulted. The same issue exists in the InGivenOutMath.quoteInGivenOutSS path used by swapExactOut.

Affected Code

- WeightedPool.sol#L523C9-L543C10

Impacts
All intent based swaps executed on pools with SmartShield enabled are charged standard swap and taker fees instead of the intended intent specific fees.

Remediation

It is suggested to pass the isIntentSwap flag through the entire SmartShield calculation chain.

Retest

This issue has been fixed by applying IntentSwap fee to SmartSHielded pools.

Bug ID #M005 [Fixed]

swapExactOut Reverse Pass Uses Router as Slicing User Instead of Actual Trader

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
In MultiHopRouter.swapExactOut(), the reverse pass computes the required input per hop using sspCore.quoteInGivenOutSS() with address(this) (the router) as the slicing user:

```
// Reverse pass
(amountInRequired,,) = pool.sspCore().quoteInGivenOutSS(
  ...,
  address(this),   // <<<< router as slicing user
  address(pool),
  ...
);
```

The forward pass passes msg.sender (the actual trader):

```
// Forward pass
pool.swapExactOut(
  ...,
  msg.sender,    // <<<< actual user as slicing user
  false
);
```

Since TradeSlicingProtection tracks per-user sessions, the sessionized WBF computed for the router's session (likely clean/empty) can differ significantly from the user's session (which may have accumulated fees from prior trades). This causes amountInRequired from the reverse pass to diverge from the actual amountIn needed in the forward pass, leading to either reverts or over-allocated input per hop.

Affected Code

- MultiHopRouter.sol#L275

Impacts

When a user with an active trade slicing session calls swapExactOut through an SSP-enabled pool, the reverse pass computes WBF for the router's clean session instead of the user's actual session. If the user's session yields higher WBF, the forward execution reverts because the actual amountIn exceeds the estimated inAmounts[i] used as maxAmountIn, causing a persistent DoS. If the user's session yields lower WBF, the per-hop input is over-allocated, leaving excess tokens unaccounted for in the router.

Remediation

Use msg.sender as the slicing user in the reverse pass to match the forward execution:

Retest

swapExactOut reverse pass now uses msg.sender as the slicing user instead of address(this) (the router), matching the forward execution-

Bug ID #M006 [Fixed]

Fallback Feeder Price in RyzeRouter.getQuote Ignores inAmount Parameter

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
The RyzeRouter.getQuote function implements the IPriceOracle interface which specifies that given an inAmount of a base token, it should return the equivalent outAmount in quote tokens. When the oracle switch is set to use the fallback oracle and the fallback oracle's price is older than the manually set feeder price, the function falls through to a branch that returns fallbackOracleFeederPrice[base] as the outAmount. This stored value is a fixed number that does not incorporate the inAmount parameter in any way.

Affected Code

- RyzeRouter.sol#L186

Impacts
Any price query through getQuote that hits the feeder fallback path returns a value completely unrelated to the queried amount.

Remediation
It is suggested to treat the stored feeder price as a unit price and scale it by inAmount.

Retest
Removed fallbackOracleFeederPrice and fallbackOracleFeederPriceUpdatedAt mappings entirely from RyzeRouter; all price paths now use direct oracle price, eliminating the inAmount-ignoring fallback

Bug ID #M007 [Fixed]

swapExactOut Slippage Check Uses Reverse-Pass Estimates Before Forward Execution

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
In MultiHopRouter.swapExactOut(), the overall slippage check on line is performed before the forward execution on lines:

```
// Reverse pass computes estimated inAmounts
...

// slippage check on ESTIMATES
if (inAmounts[0] > params.maxAmountIn) revert SlippageExceeded();

// actual forward execution happens AFTER the check
for (uint256 i = 0; i < n; i++) {
  pool.swapExactOut(...);
}
```

The reverse pass uses view-only quoteInGivenOutSS() calls to estimate the required input per hop. These estimates can diverge from the actual amounts consumed during forward execution (due to state changes, different trade slicing sessions, etc.). By performing the slippage check on these estimates rather than on the actual execution results, the check can falsely reject valid transactions where the estimate overshoots but the actual execution would have been within the user's tolerance.

Affected Code

- MultiHopRouter.sol#L295

Impacts
The premature slippage check against reverse-pass estimates causes valid transactions to revert when the view-only quote overestimates the required input amount, resulting in a denial of service

for swapExactOut users on SSP-enabled pools where the actual forward execution would have succeeded within the user's specified maxAmountIn.

Remediation

Move the slippage check after the forward execution, using the actual consumed amount from pool.swapExactOut().

Retest

swapExactOut slippage check now uses actual consumed amount from pool.swapExactOut() return value instead of reverse-pass estimates

Bug ID #M008 [Fixed]

updateConfigBatch Validates Old Config Instead of New Config Before Storage

Vulnerability Type
Lack of Input Validation (SC04-Lack Of Input Validation)

Severity
Medium

Description
In WeightBreakingRewardConfig.sol, the updateConfigBatch function is intended to replace the entire configuration struct with a new one provided by the caller. The function calls _validateConfig() before assigning the new config to storage. However, _validateConfig() is an internal view function that reads exclusively from _config, the current state variable containing the existing configuration. At the time _validateConfig() executes, the newConfig parameter has not yet been written to _config, so every validation check operates on the old configuration values that are about to be discarded.

The function body executes in this order: first, _validateConfig() reads and validates _config (the current stored config), confirming it satisfies all invariants. Then _config = newConfig writes the unvalidated newConfig directly to storage, overwriting the config that was just validated. The validation pass provides no protection whatsoever because it checks values that are not being changed.

Affected Code

- WeightBreakingRewardConfig.sol#L232C5-L239C6

Impacts
A privileged caller with the config role can unintentionally set arbitrary configuration values that violate all safety invariants without any revert.

Remediation
It is recommended to change _validateConfig() to accept the config struct as a parameter and validate the passed in struct rather than reading from storage.

Retest

updateConfigBatch() in WeightBreakingRewardConfig now validates the newConfig parameter before writing to storage, instead of validating the stale existing config

Bug ID #M009 [Fixed]

WBR Tokens Permanently Stuck in Router During swapExactOut Forward Execution

Vulnerability Type
Stuck Funds

Severity
Medium

Description
In MultiHopRouter.swapExactOut(), the forward execution calls pool.swapExactOut() with address(this) (the router) as the recipient. When a swap improves pool balance, the pool awards a Weight Breaking Reward (WBR) to the recipient via _collectAndDistributeFees(feeDetails, to). These WBR tokens are sent to the router but are never forwarded to the user or accounted for in subsequent hops.

In contrast, swapExactIn correctly handles WBR by capturing the fee details and adding the WBR amount to the output for the next hop:

```
// swapExactIn WBR correctly added to next hop
if (feeDetails.wbrFee.token == hop.tokenOut && feeDetails.wbrFee.amount > 0) {
  wbrFeeAmount = feeDetails.wbrFee.amount;
}
currentAmount = hopAmountOut + wbrFeeAmount; // ✅ user benefits
```

```
// swapExactOut return values not captured
pool.swapExactOut(
  hop.tokenIn, hop.tokenOut, outAmounts[i], inAmounts[i],
  address(this), pairCexPrice, params.path.length, msg.sender, false
);
// ❌ WBR sent to router, never forwarded
```

Since the router has no withdrawal function for arbitrary tokens, WBR tokens that accumulate in the router are permanently locked.

Affected Code

- MultiHopRouter.sol#L201

Impacts

Every swapExactOut through an SSP-enabled pool where the trade improves pool balance results in WBR tokens being permanently locked in the router. The user who triggered the improving trade receives no reward, and the locked tokens are irrecoverable since the router has no token rescue mechanism.

Remediation

Capture the fee details from pool.swapExactOut() and forward WBR tokens to the user.

Retest

WBR tokens in swapExactOut forward execution are now forwarded to the recipient; _collectAndDistributeFees returns actualWbrDistributed for proper handling

Bug ID #M010 [Fixed]

swapExactOut Refund Uses Reverse-Pass Estimate Instead of Actual Amount Consumed

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
In MultiHopRouter.swapExactOut(), the user provides params.maxAmountIn tokens upfront . After the forward execution, the router refunds unused tokens on:

```
//ser sends max tokens upfront
IERC20(params.tokenIn).safeTransferFrom(msg.sender, address(this),
params.maxAmountIn);

// forward execution (return values not captured)
pool.swapExactOut(..., outAmounts[i], inAmounts[i], ...);

// refund based on REVERSE-PASS estimate
uint256 remaining = params.maxAmountIn - inAmounts[0];
```

inAmounts[0] is computed during the reverse pass using view-only quote functions. The actual amountIn consumed by pool.swapExactOut() during forward execution can differ (due to trade slicing sessions, WBF/WBR variations, etc.), but its return value is never captured. The refund always uses the stale reverse-pass estimate.
When the actual forward-pass consumption is less than inAmounts[0], the user is under-refunded the difference remains permanently stuck in the router. For example:

- params.maxAmountIn = 1000
- Reverse-pass estimate: inAmounts[0] = 950
- Actual forward consumption: 900
- Refund: 1000 - 950 = 50 (should be 1000 - 900 = 100)
- 50 tokens stuck in router

Affected Code

- MultiHopRouter.sol#L329

Impacts

Users calling swapExactOut through SSP-enabled pools where the actual forward execution consumes less input than the reverse-pass estimate will be under-refunded, with the difference permanently locked in the router. This constitutes a direct loss of funds for the user with no recovery mechanism.

Remediation

Capture the actual amountIn from the first hop's pool.swapExactOut() and use it for the refund.

Retest

Refund in swapExactOut now uses actual consumed amount from the first hop's pool.swapExactOut() return value instead of the stale reverse-pass estimate

Bug ID #M011 [Acknowledged]

Missing Minimum Weight Enforcement in Basic (Non SmartShield) Swap Path

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
Medium

Description
The WeightedPool contract enforces a minimum weight cap of 15% (MIN_WEIGHT_WAD = 15e16) when SmartShield is enabled. This cap prevents any single swap from pushing the pool's oracle derived weight ratio beyond 85/15, protecting against extreme imbalance that could drain one side of the pool and create catastrophic pricing conditions. The enforcement occurs inside WeightedPoolMath.oracleNormalizedWeightsWithDecimals(), which is called during the SmartShield swap path through OutGivenInMath.quoteOutGivenInSS() and through weightFees(). If a swap would push either asset's weight below 15%, the function reverts with CapReached().

When SmartShield is disabled, _executeSwapExactIn and _executeSwapExactOut take the basic path, calling quoteOutGivenInBasic or quoteInGivenOutBasic respectively. These functions use the statically configured weights (inPa.weight, outPa.weight) directly in the Balancer AMM formula without computing oracle derived weights and without checking whether the resulting pool state would violate the 15% minimum weight invariant. The only validation after the basic swap calculation is that amountOut > 0, that slippage is acceptable, and that enough liquidity exists (outPa.balance < amountOut). None of these checks constrain the post swap weight distribution.

Affected Code

- WeightedPool.sol#L541C9-L544C1

Impacts
Pools operating without SmartShield have no protection against extreme weight imbalance from swaps. A trader or series of traders can progressively drain one side of the pool without triggering any safety limit, pushing the effective weight ratio past the 85/15 boundary that the protocol considers unsafe.

Remediation

It is recommended to add a post swap weight validation in both _executeSwapExactIn and _executeSwapExactOut that applies regardless of whether SmartShield is enabled.

Retest

Client's Comment:- for basic pools we don't need extreme weights checks

Bug ID #M012 [Fixed]

calculateSessionizedFees Ignores AMM-Computed Slippage and WBF, Uses Simplified Formula Instead

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
TradeSlicingProtection.calculateSessionizedFees() accepts currentSlippage and currentWbf as parameters these are the actual fees computed by the AMM math libraries (OutGivenInMath/InGivenOutMath) which account for external liquidity ratios, band selection, fee suppression, and oracle-aware weights:

```
// OutGivenInMath carefully computes slippage using bands, external liquidity, etc.
uint256 finalSlippageAmountUsd = (normalizedFinalSlippage * pOut) / WAD;
uint256 initialWbfAmountUsd = (normalizedInitialWbf * pIn) / WAD;

// Passes them to calculateSessionizedFees
sessionResult = tradeSlicingProtection.calculateSessionizedFees(
  ...,
  finalSlippageAmountUsd,  // currentSlippage IGNORED
  initialWbfAmountUsd,   // currentWbf IGNORED
  ...
);
```

However, _calculateSessionizedFees never uses these parameters. Instead, it recalculates fees from scratch using simplified formulas:
Slippage _calculateTotalSlippageForNotional :

```
// Simplified: Slip = ((ExecPrice / startPrice) - 1) × Notional
uint256 slippageRate = priceImpact - WAD;
return (slippageRate * notional) / WAD;
```

WBF _calculateTotalWbfForNotional:

```
// Simplified: uses computeWBFCharge with currentWeight only
uint256 wbfRate = weightBreakingReward.computeWBFCharge(
```

```
  notional, UD60x18.wrap(currentWeight), ...
);
```

The simplified slippage formula uses only execPrice / startPrice and ignores external liquidity ratios, band-specific slippage floors, and fee suppression all of which the real AMM math applies. For example, a trade with externalLiquidityRatio = 50e18 (50x external liquidity) would compute very small slippage in OutGivenInMath, but _calculateTotalSlippageForNotional would compute a much larger value since it doesn't model external liquidity at all.

Affected Code

- TradeSlicingProtection.sol#L464

Impacts

The sessionized fees can diverge significantly from the actual AMM-computed fees because the simplified formulas omit key factors (external liquidity ratios, band selection, fee suppression). When the simplified formula overestimates fees, users are charged more than the AMM would charge for an equivalent single-shot trade, making the slicing protection excessively punitive. When it underestimates, an attacker can slice trades to pay less cumulative WBF/slippage than a single large trade would incur, defeating the purpose of trade slicing protection.

Remediation

Use the AMM-computed currentSlippage and currentWbf values in the sessionization logic rather than recalculating from scratch. The incremental approach should accumulate actual AMM fees and compare against what a single-shot trade of the cumulative notional would pay using the same AMM math

Retest

_calculateSessionizedFees now uses external slippageQuery contract (e.g., WeightedPoolQueries.getSlippageForNotional) for on-curve slippage computation; AMM-computed slippage used as floor via max(sessionized, currentSlippage)

Bug ID #M013 [Fixed]

_calculateSessionizedFees Uses Undecayed Notional for Previous Fee Baseline

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
In TradeSlicingProtection._calculateSessionizedFees(), the "new" cumulative notional correctly applies EMA decay before adding the current trade, but the "previous" baseline fees are calculated on the raw, undecayed session.cumulativeNotional:

```
// NEW total uses decayed notional ✅
uint256 decayedNotional = _applyEmaDecay(session.cumulativeNotional, ...);
uint256 newTotalNotional = decayedNotional + tradeNotional;

// Fees for NEW total based on decayed + trade ✅
uint256 totalSlippageForCumulative = _calculateTotalSlippageForNotional(newTotalNotional,
...);
uint256 totalWbfForCumulative = _calculateTotalWbfForNotional(newTotalNotional, ...);

// Fees for PREVIOUS based on RAW undecayed ❌
uint256 totalSlippageForPrevious =
_calculateTotalSlippageForNotional(session.cumulativeNotional, ...);
uint256 totalWbfForPrevious = _calculateTotalWbfForNotional(session.cumulativeNotional,
...);

// Incremental = new - previous
sessionizedSlippage = totalSlippageForCumulative > totalSlippageForPrevious
  ? totalSlippageForCumulative - totalSlippageForPrevious : 0;
```

Since session.cumulativeNotional > decayedNotional (decay always reduces), the previous baseline is inflated. When the decay is significant (many blocks elapsed), it's possible that session.cumulativeNotional > newTotalNotional, making totalForPrevious > totalForCumulative, which zeros out the sessionized fees entirely.

Example:

session.cumulativeNotional = 100,000 USD (raw)
emaAlpha = 0.9e18, 10 blocks elapsed → decayedNotional = 34,868 USD
tradeNotional = 5,000 USD
newTotalNotional = 34,868 + 5,000 = 39,868 USD

Affected Code

- [TradeSlicingProtection.sol#L485-L488]

Impacts

After blocks elapse within a session, trades can pay zero sessionized WBF and slippage because the inflated previous baseline exceeds the new decayed total. This directly undermines trade slicing protection; an attacker can wait a few blocks between sliced trades to ensure the decay makes the previous baseline dominate, effectively paying single-trade fees instead of cumulative session fees.

Remediation

Use decayedNotional for the previous fee baseline to maintain consistency.

Retest

TradeSlicingProtection rewritten with signed netFlow model and _applyEmaDecayInt; the undecayed vs decayed baseline inconsistency is resolved in the new architecture

Bug ID #M014 [Fixed]

Sessionized Slippage Is Computed But Never Applied to Adjust Swap Output

Vulnerability Type
Dead Code

Severity
Medium

Description
TradeSlicingProtection._calculateSessionizedFees() computes both sessionizedSlippage and sessionizedWbf. However, only sessionizedWbf is used by the swap math libraries to adjust the final swap output. The sessionizedSlippage follows a dead path:

```
_calculateSessionizedFees()
 → returns (sessionizedSlippage, sessionizedWbf)
  → sessionResult.sessionizedSlippage    // stored in result

OutGivenInMath / InGivenOutMath:
 → sessionResult.sessionizedWbf      // ✅ extracted and applied
 → sessionResult.sessionizedSlippage     // ❌ never read

SSPCore.handleSSPoolSwap():
 → tradeSlicingProtection.updateSessionState(
   ..., sessionResult.sessionizedSlippage, ...) // stored in session
 → session.cumulativeSlippage += sessionizedSlippage // accumulated but never queried
```

The actual slippage applied to the swap is computed independently by OutGivenInMath/InGivenOutMath using external liquidity ratios, band selection, and fee suppression and is never replaced or adjusted by the sessionized value. This means trade slicing protection only protects against WBF splitting, not slippage splitting.

Affected Code

- OutGivenInMath.sol#L237-L238
- InGivenOutMath.sol#L235-L236
- TradeSlicingProtection.sol#L262

Impacts

Trade slicing protection is only half-effective it sessionizes WBF but not slippage. An attacker can split a large trade into multiple smaller trades within a session to pay less cumulative slippage than a single-shot trade would incur, since each small trade independently computes its own slippage without session-level adjustment. The entire sessionizedSlippage computation and cumulativeSlippage storage are wasted gas with no functional effect.

Remediation

Apply sessionizedSlippage to adjust the swap's slippage, similar to how sessionizedWbf replaces the initial WBF.

Retest

sessionResult.sessionizedSlippage is now applied in both OutGivenInMath and InGivenOutMath to compute finalSlippageAmount and adjust swap output/input

Bug ID #M015 [Fixed]

_executeSwapExactOut Checks Input Token Balance Instead of Output Token Balance

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
In WeightedPool._executeSwapExactOut(), after computing the required amountIn, there is a post-calculation liquidity check:

```
if (inPa.balance < amountIn) revert InsufficientLiquidity();
```

This is incorrect. In a swapExactOut:

The pool receives tokenIn from the caller (safeTransferFrom) no need for the pool to already hold this amount
The pool sends tokenOut to the recipient (safeTransfer) this is where liquidity matters
The check should verify output token balance, not input. A correct pre-calculation check already exists earlier:

```
if (outPa.balance < amountOut) revert InsufficientLiquidity(); // ✅ correct, before calculation
```

However, the post-calculation check on inPa.balance can cause false reverts when the pool holds a small balance of tokenIn but is receiving a large incoming amount a perfectly valid swap that should succeed.

Affected Code

- WeightedPool.sol#L636

Impacts
Valid swapExactOut transactions revert when the pool's existing tokenIn balance is less than the computed amountIn, even though the pool is receiving (not spending) tokenIn. This causes a denial of service for swaps involving tokens where the pool holds a small balance on the input side, which

is a normal and expected pool state particularly after the pool has been drained of tokenIn through prior swaps in the opposite direction.

Remediation

Remove the incorrect check since the correct output liquidity check already exists before the calculation Or, if a post-calculation check is desired for defense-in-depth, check the correct token.

Retest

_executeSwapExactOut now correctly checks outPa.balance < amountOut (output token balance) instead of the incorrect inPa.balance < amountIn (input token balance)

Bug ID #M016 [Fixed]

createPool Allows Arbitrary Oracle and WBR Contracts Without Whitelisting

Vulnerability Type
Access Control (SCWE-016)

Severity
Medium

Description
WeightedPoolFactory.createPool() has no access control and accepts user-supplied oracle and weightBreakingReward addresses without any validation beyond a zero-address check:

```
function createPool(
  string memory name,
  string memory symbol,
  address oracle,        // <<<< user-supplied
  address weightBreakingReward,  // <<<< user-supplied
  WeightedPool.AssetConfig[ ] memory assets,
  bytes memory paramsData,
  address admin        // <<<< user-supplied
) external returns (WeightedPool pool) {
  require(oracle != address(0), "FACTORY: Invalid oracle");
  require(weightBreakingReward != address(0), "FACTORY: Invalid weightBreakingReward");
  // No whitelist check any contract accepted
}
```

The factory registers every created pool as legitimate via isPool[address(pool)] = true, regardless of the contracts passed. This is inconsistent with how equally critical infrastructure contracts are handled:tradeSlicingProtection and sspCore.

A malicious oracle can manipulate all SmartShield pricing slippage, oracle weights, WBF calculations. A malicious weightBreakingReward can return inflated or zero WBF/WBR fees. Both are registered under isPool, making them appear legitimate to downstream contracts like MultiHopRouter.

Additionally, the factory has no duplicate pool prevention there is no check for whether a pool with the same token pair already exists. An attacker can create a copycat pool with the same assets but a malicious oracle, which appears identical to the legitimate pool in the factory registry.

Affected Code

- WeightedPoolFactory.sol#L109

Impacts

Anyone can deploy a pool with a malicious oracle or WBR contract that is registered as a legitimate factory pool. If downstream systems (routers, aggregators, UI) trust the factory's isPool registry to determine pool legitimacy, users can be routed through these malicious pools and suffer manipulated pricing, inflated fees, or suppressed slippage protection.

Remediation

Maintain a whitelist of approved oracle and weightBreakingReward contracts at the factory level, preserving permissionless pool creation.

Retest

Added isApprovedOracle and isApprovedWeightBreakingReward whitelists in WeightedPoolFactory; added poolByTokenPair mapping for duplicate pool prevention; pool registration with SSPCore.

Bug ID #M017 [Fixed]

Fee Suppression Restores Swap Fee on Improving Trades That Should Pay Zero

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
In both OutGivenInMath and InGivenOutMath, weightFees returns chargeSwapFee = false for improving trades that exceed the thresholdWeightDiffSwapFeeWad. The swap fee is correctly set to zero:

```
uint256 swapFeeWad = chargeFee ? params.swapFeeWad : 0;  // → 0 for improving trades
```

However, weightFees also returns shouldApplyFeeSuppression = true for all improving trades. The fee suppression block unconditionally overwrites swapFeeWad:

```
if (shouldApplyFeeSuppression) {
  UD60x18 sigma = weightBreakingReward.sigmaSuppression(UD60x18.wrap(wStart));
  uint256 sigmaRaw = UD60x18.unwrap(sigma);

  uint256 suppressedSwapFeeWad = (sigmaRaw * params.swapFeeWad) / WAD;
  swapFeeWad = suppressedSwapFeeWad;  // <<<< restores to non-zero!
}
```

The suppressedSwapFeeWad is computed from params.swapFeeWad (not from the current swapFeeWad), so it ignores the earlier chargeSwapFee = false decision. An improving trade that was supposed to pay zero swap fee ends up paying sigma * params.swapFeeWad.
Trigger conditions (from weightFees):

- diff < 0 (improving trade) → chargeSwapFee = false (if initDist > thresholdWeightDiffSwapFeeWad)
- shouldApplyFeeSuppression = true (always set for improving trades)
- Both conditions true simultaneously → swap fee zeroed then restore

Affected Code

- InGivenOutMath.sol#L181C8-L196C10
- OutGivenInMath.sol#L183C7-L199C1

Impacts

Users who perform improving trades (rebalancing the pool) and qualify for zero swap fee are still charged a suppressed swap fee, reducing their output. This penalizes exactly the behavior the protocol should incentivize users who help restore pool balance receive less favorable pricing than intended.

Remediation

The fee suppression block should compute suppressedSwapFeeWad from the current swapFeeWad (which is already zero when chargeSwapFee = false) instead of from params.swapFeeWad (the original configured value). This ensures the suppression respects the earlier chargeSwapFee decision suppressing zero remains zero, while suppressing the full fee for worsening trades still works correctly. The same fix should be applied in both OutGivenInMath and InGivenOutMath.

Retest

Fee suppression logic restructured in OutGivenInMath and InGivenOutMath: suppression applied first (reducing fees via sigma), then a separate if (!chargeFee) block zeros out swap fee, preventing the restore-after-zero bug-

Bug ID #M018 [Fixed]

Incorrect Slippage Fee Calculation in Basic Swap Path (quoteOutGivenInBasic and quoteInGivenOutBasic)

Vulnerability Type
Accounting / Incorrect Calculation

Severity
Medium

Description
The slippage fee calculation in both SwapOperations.quoteOutGivenInBasic and SwapOperations.quoteInGivenOutBasic contains three compounding errors that produce an inaccurate slippageFee.amount in the returned FeeDetails:

1. Missing weight terms in spot price: The spot price is calculated as balanceOut / balanceIn, but the correct Balancer weighted pool spot price is (balanceOut / balanceIn) × (weightIn / weightOut). The weight ratio is entirely omitted. This produces a correct result only for 50/50 pools; for any non-equal-weight pool (e.g., 80/20), the spot price is fundamentally wrong.

2. Pre-fee amountIn used in effective price: amountOut is computed from amountInAfterFee (fee-reduced input via BalancerMath.outGivenIn), but effectivePriceOutPerIn divides amountOut by the raw amountIn (pre-fee). This mixes two different input quantities, causing the fee portion to be misattributed as slippage and inflating slipAmt by the fee percentage.

3. Price difference treated as token amount: slipAmt represents a per-unit price difference (output tokens per input token), not a total token amount. However, it is passed directly to denormalizeBalance as if it were a token amount. The correct total slippage in output tokens requires multiplying the per-unit price difference by the number of input units traded: slippageTokens = (spotPrice − effectivePrice) × amountInAfterFee.

```
uint256 normalizedAmountIn = WeightedPoolMath.normalizeBalance(
  amountIn,                <<<<<<< pre-fee amount used
  inPa.decimals
);

uint256 spotPriceOutPerIn = FixedPointMathLib.mulDivDown(
```

```
  normalizedBalanceOut, WAD, normalizedBalanceIn
);                          <<<<<<< missing weight terms

uint256 effectivePriceOutPerIn = FixedPointMathLib.mulDivDown(
  normalizedAmountOut, WAD, normalizedAmountIn
);

if (spotPriceOutPerIn > effectivePriceOutPerIn) {
  uint256 slipAmt = spotPriceOutPerIn - effectivePriceOutPerIn;
  feeDetails.slippageFee.amount = WeightedPoolMath
    .denormalizeBalance(slipAmt, outPa.decimals);          <<<<<<< price diff directly used as
token amount

}
```

Affected Code

- SwapOperations.sol#L77
- SwapOperations.sol#L26

Impacts

The inaccurate slippageFee.amount is passed to _collectAndDistributeFees(), which distributes tokens to protocol, distribution, and treasury addresses based on configured ratios. Since slippage in the basic path is already embedded in the Balancer curve output and is never separately deducted from amountOut, any tokens distributed as "slippage fees" are drawn from pool balances that belong to liquidity providers, resulting in a direct LP fund loss. The error is significantly amplified for non-50/50 pools due to the missing weight terms, while simultaneously being understated in absolute terms due to the missing multiplication by trade size.

Remediation

The spot price calculation should include the weight ratio as (balanceOut / balanceIn) × (weightIn / weightOut). The effective price should use amountInAfterFee instead of the raw amountIn so that only pure AMM price impact is measured. The per-unit price difference must be multiplied by amountInAfterFee to convert it into a total slippage amount in output tokens. The same corrections should also be verified in quoteInGivenOutBasic, which has a similar slippage calculation pattern.

Retest

Spot price in SwapOperations now includes weight ratio (weightIn/weightOut); slippage uses amountInAfterFee instead of pre-fee amount; per-unit price difference properly multiplied by trade size

Bug ID #M019 [Fixed]

Perpetual Emergency Lockout via Cooldown Reset

Vulnerability Type
Governance DoS

Severity
Medium

Description
The emergencyPause() function resets the emergencyCooldown timestamp on every invocation without checking if the system is already paused. Since emergencyUnpause() enforces block.timestamp >= emergencyCooldown before allowing unpause, an EMERGENCY_ROLE holder can repeatedly call emergencyPause() before the existing cooldown expires, pushing the cooldown window forward indefinitely.

This creates a state-machine flaw where governance is permanently locked out from unpausing the system, resulting in a complete protocol availability denial-of-service.

Affected Code

- MerkleDistributor.sol#L169-L186

Impacts
Permanent protocol pause where governance can never satisfy the emergencyCooldown requirement. All whenNotPaused operations (e.g., submitEpochRoot) are blocked indefinitely. No fund theft occurs, but protocol liveness is fully compromised. Recovery requires emergency key revocation, governance override, or contract upgrade.

Remediation
Add a guard to prevent cooldown reset when the system is already paused.

Retest
This issue has been fixed.

Bug ID #M020 [Fixed]

Missing Solvency Invariant Allows Permanent Claim Failure

Vulnerability Type
Centralization / Accounting Risk

Severity
Medium

Description
The contract does not enforce the solvency invariant that the USDC balance held must always be greater than or equal to the outstanding cumulative liability defined as totalDistributedSum - totalClaimedCumulative. If the actual token balance falls below this liability, valid user claims will revert with InsufficientBalance(), even when a correct Merkle proof is provided.

The accounting system increases totalDistributedSum before verifying the exact amount of tokens received. If the token has non-standard behavior or fewer tokens are received than expected, the internal accounting can diverge from the real balance. Additionally, governance can withdraw USDC through emergencyWithdraw without ensuring that enough funds remain to satisfy outstanding liabilities. Once the contract becomes underfunded, users with legitimate rewards may be permanently unable to claim unless additional tokens are deposited

Affected Code

- RevenueDistributor.sol#L125-L157
- RevenueDistributor.sol#L159-L203

Impacts
If insolvency occurs, valid claims permanently revert despite correct proofs. This results in a denial of service for affected users and creates a mismatch between recorded liabilities and actual token reserves, undermining system reliability.

Remediation
Verify that the actual received token amount matches the declared distribution before updating accounting values. Additionally, enforce a solvency check in emergency withdrawals to ensure that the remaining balance always covers outstanding liabilities.

Retest

This issue is fixed as submitRevenueRoot() now snapshots USDC.balanceOf(address(this)) before and after safeTransferFrom, reverting with ReceivedAmountMismatch if received amount differs from declared, so totalDistributedSum only advances on verified receipt. emergencyWithdraw() enforces balance >= amount + (totalDistributedSum - totalClaimedCumulative) before any transfer, preventing governance from draining below the outstanding claim liability.

Bug ID #M021 [Fixed]

Permanent Reward Lock on Token Removal

Vulnerability Type
Governance-Controlled Funds Lock

Severity
Medium

Description
When a reward token is removed via removeRewardToken(), the contract immediately prevents any further claims for that token. The claimRewards() function enforces a whitelist check through the if (!whitelistedRewardTokens[token]) revert TokenNotWhitelisted() guard, which causes all claim attempts for the removed token to revert.

Additionally, _harvestAndSyncRewardDebt() only iterates over the current whitelistedRewardTokenList array, so the removed token is no longer processed during stake or unstake operations. This means that any rewards a user legitimately earned before the token was removed become permanently inaccessible.

The accumulated accRewardPerShare and per-user rewardDebt and pendingClaim state for the removed token remain in storage but can never be settled. The only path to recovery is for governance to re-whitelist the token, which reintroduces whatever risk prompted the removal in the first place.

Affected Code

- LPGaugeVault.sol#L295-L310
- LPGaugeVault.sol#L244
- LPGaugeVault.sol#L185-L203

Impacts
Users permanently lose access to legitimately accrued rewards for a token once it is removed from the whitelist, creating a governance-controlled funds lock where earned rewards become unreachable despite remaining recorded in contract storage.

Remediation

Allow claiming of previously accrued rewards even after token removal by removing the whitelist check in claimRewards() for historical balances or introducing a dedicated claim path for de-whitelisted tokens before fully disabling the reward token.

Retest

This issue has been fixed as claimRewards() now gates on _isClaimableRewardToken() instead of a raw whitelist check, this helper returns true for any token where rewardIndices[token].lastUpdateTime != 0, meaning all tokens that ever had rewards distributed remain claimable by users even after governance removes them from the active whitelist.

Bug ID #M022 [Fixed]

Unbounded Reward Token List May Cause Gas-Based DoS

Vulnerability Type
Gas Denial of Service

Severity
Medium

Description
The _harvestAndSyncRewardDebt() function iterates through every entry in the whitelistedRewardTokenList array during every stake(), unstake(), and implicit harvest operation. For each token in the list, the function reads the user's UserRewardState from storage, reads the token's RewardIndex from storage, computes pending rewards via calculatePendingRewards(), and if pending rewards are non-zero, performs an external balanceOf() call, an external safeTransfer() call, a second balanceOf() call, and a storage write to update the user's reward state.

Gas consumption therefore grows linearly with the number of whitelisted reward tokens. There is no upper bound enforced on the length of whitelistedRewardTokenList in the whitelistRewardToken() function any number of tokens can be added as long as the caller holds ADMIN_ROLE.

If a sufficiently large number of reward tokens are whitelisted, the cumulative gas cost of iterating and performing external calls for each token during a single stake() or unstake() transaction can approach or exceed the block gas limit, causing the transaction to revert. Since both staking and unstaking are gated behind this iteration, an excessive reward token list effectively freezes all user interactions with the vault.

Affected Code

- LPGaugeVault.sol#L185-L203
- LPGaugeVault.sol#L285-L294

Impacts
If the reward token list becomes too large, stake() and unstake() can exceed block gas limits due to the unbounded harvest loop, causing transactions to revert and temporarily preventing users from depositing or withdrawing their LP tokens, effectively resulting in a vault-wide denial of service.

Remediation

Enforce a strict maximum limit on whitelisted reward tokens or refactor harvesting to a per-token claim model so stake() and unstake() do not iterate over an unbounded reward list.

Retest

This issue has been fixed as MAX_WHITELISTED_REWARD_TOKENS = 32 is enforced in whitelistRewardToken() with revert MaxRewardTokensReached(), capping the whitelistedRewardTokenList array length and therefore the harvest loop iteration count on every _stake() and unstake() call to a fixed maximum of 32.

Bug ID #M023 [Fixed]

Reward Debt Desynchronization After Token Removal and Re-Whitelisting

Vulnerability Type
State Management Flaw

Severity
Medium

Description
When a reward token is removed via removeRewardToken(), its entry is deleted from whitelistedRewardTokenList and whitelistedRewardTokens[token] is set to false. However, the per-user rewardDebt and pendingClaim values stored in userRewardStates[user][token] remain unchanged in storage. If users subsequently modify their LP stake by calling stake() or unstake(), the _harvestAndSyncRewardDebt() function only iterates over the current whitelistedRewardTokenList and therefore skips updating rewardDebt for the removed token entirely.

This means each user's rewardDebt for that token remains frozen at the value it held at the time of removal, tied to whatever lpAmount the user had at that point. If governance later re-whitelists the same token via whitelistRewardToken(), the token re-enters the iteration loop but each user's rewardDebt is still based on their oldlpAmount rather than their current stake. The calculatePendingRewards() function computes pending rewards as (currentLpAmount * accRewardPerShare / PRECISION) - rewardDebt, and since currentLpAmount may have changed significantly while rewardDebt remained stale, the resulting calculation produces incorrect values.

Users who increased their stake during the removal window will have an inflated accumulated figure against an outdated debt, receiving more rewards than they are entitled to. Conversely, users who decreased their stake will have a deflated accumulated figure, potentially receiving zero or far fewer rewards than earned.

Affected Code

- LPGaugeVault.sol#L295-L310
- LPGaugeVault.sol#L185-L203

Impacts
Reward calculations become corrupted after a token is re-whitelisted following a removal. Users who modified their stake during the removal window will experience incorrect reward payouts

either receiving excess rewards they did not earn or losing access to rewards they legitimately accrued.

This breaks the core reward accounting invariants of the MasterChef-style system, where rewardDebt must always reflect the user's current lpAmount multiplied by accRewardPerShare. The desynchronization can also cascade into the vault's overall reward balance, potentially draining reward reserves faster than intended or leaving undistributed tokens permanently stranded in the contract.

Remediation

Disallow re-whitelisting of previously removed tokens, or ensure full state synchronization before reactivation by updating each user's rewardDebt to match their current lpAmount and the latest accRewardPerShare; alternatively, mark tokens as inactive instead of removing them so accounting updates continue even while distribution is paused.

Retest

This issue has been fixed as removeRewardToken() permanently sets removedRewardTokens[token] = true, and whitelistRewardToken() reverts with TokenPreviouslyRemoved() if this flag is set on the candidate token. Because re-whitelisting is unconditionally and irrevocably blocked, the rewardDebt desynchronization attack path cannot be triggered.

Bug ID #M024 [Fixed]

removeActivity Reverts on Zero-Weight LP Pools While removeActivityFromTokens Silently Succeeds

Vulnerability Type
Inconsistent Validation

Severity
Medium

Description
removeActivity and removeActivityFromTokens handle zero-weight LP pools inconsistently.
When _applyPoolWeight() returns 0 (due to poolWeights[msg.sender] == 0, an explicitly set zero weight, or rounding via mulWadDown), removeActivity forwards the zero value to _removeActivityInternal, which reverts with InvalidActivity().

In contrast, removeActivityFromTokens checks if (adjustedAmount > 0) and silently returns if the value is zero. As a result, identical zero-weight conditions cause one function to revert and the other to succeed, creating inconsistent system behavior.

Affected Code

- GaugesManager.sol#L197-L213
- GaugesManager.sol#L253-L291
- GaugesManager.sol#L291-L322
- GaugesManager.sol#L828-L846

Impacts
If an integrating contract (e.g., during LP exit flows) calls removeActivity, any pool with unset or zero weight will cause the transaction to revert, potentially blocking liquidity withdrawals and creating a denial-of-service condition. Since pools default to weight 0 and weight-setting is not enforced atomically with authorization, this misconfiguration scenario is realistic.

Remediation
Add a zero-value guard in removeActivity to match removeActivityFromTokens, or enforce non-zero pool weights during authorization so zero-weight LP pools cannot trigger unexpected reverts.

Retest

This issue has been fixed by removing the vulnerable file.

Bug ID #M025 [Fixed]

Oracle Price Drift Causes Phantom Activity and Unfair Emission Allocation

Vulnerability Type
Business Logic Flaw

Severity
Medium

Description
markActivityFromTokens and removeActivityFromTokens convert token amounts to USD using the oracle's current spot price at the time of each call. The USD value is stored directly in epochUserActivity and epochTotalActivity, while the original token amount and conversion price are not preserved.

If the token price decreases between marking and removal, the USD value calculated during removal will be smaller than the value recorded during marking. Because removal subtracts only the current USD equivalent, a residual balance ("phantom activity") remains even though the user has fully exited their LP position.

If the price increases, the contract safely clamps the subtraction to the recorded amount, preventing underflow. Therefore, the issue is asymmetric and occurs only when prices decrease.

Affected Code

- GaugesManager.sol#L214-L253
- GaugesManager.sol#L254-L323

Impacts
Users can retain phantom activity after fully exiting their LP position if token prices fall between deposit and withdrawal. This allows them to receive emissions without maintaining actual liquidity exposure, diluting rewards for honest participants and enabling unfair emission capture within the same epoch.

Remediation
Avoid storing activity purely as spot-priced USD deltas. Instead, record activity in token-denominated terms and convert to USD only at emission calculation time, or store and reuse

the original conversion rate during removal to ensure mark/remove symmetry and prevent phantom balances.

Retest

This issue has been fixed by removing the vulnerable file.

Bug ID #M026 [Fixed]

Inverted Slippage Comparison in InGivenOutMath Disables Slippage for ExactOut SmartShield Swaps

Vulnerability Type
Logic Error

Severity
Medium

Description
In InGivenOutMath.quoteInGivenOutSS, the slippage calculation compares oracleInResized > balancerInForResized, but the correct condition is balancerInForResized > oracleInResized. For inGivenOut, the Balancer curve always demands more input than the oracle price due to price impact, meaning balancerInForResized >= oracleInResized under normal conditions. The current inverted check is almost never true, causing localSlippageAmount to remain zero for all ExactOut SmartShield swaps.

The equivalent check in OutGivenInMath (ExactIn path) correctly uses oracleOutResized > balancerOutForResized, which is the right direction because the oracle gives more output than the curve. The directions are opposite because ExactIn slippage reduces output while ExactOut slippage increases input.

Affected Code

- InGivenOutMath.sol#L100C7-L102C10

Impacts
All ExactOut swaps through the SmartShield path execute with zero slippage regardless of trade size, as the slippage floor (minSlippageWad) compares against zero and only applies if configured. Users receive better-than-fair pricing on ExactOut swaps at the expense of liquidity providers.

Remediation
Reverse the comparison and subtraction order to match the ExactOut direction.

Retest
Slippage comparison in InGivenOutMath inverted from oracleInResized > balancerInForResized to correct balancerInForResized > oracleInResized

Bug ID #L001 [Fixed]

totalFees Always Returns Zero in MultiHopRouter::_executeSwap()

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
Low

Description
In MultiHopRouter::_executeSwap(), the return variable totalFees is initialized to 0 and is never updated anywhere within the function body. On each iteration of the swap loop, pool.swapExactIn() returns a WeightedPoolMath.FeeDetails struct containing the full breakdown of fees charged during that hop swap fee, taker fee, weight breaking fee, slippage fee, and weight breaking reward. However, none of these fee values are ever accumulated into the totalFees variable. The feeDetails struct is only partially consumed: the WBR (weight breaking reward) amount is extracted and added to currentAmount for the next hop, but the remaining fee fields are completely discarded. As a result, totalFees is always returned as 0 regardless of how many fees were actually charged across all hops.

The always-zero totalFees is then used by the caller swapExactIn(), where it is passed directly into the MultiHopSwap event emission. This means every MultiHopSwap event recorded on-chain reports totalFees = 0, making the fee data completely unreliable for any off-chain analytics, dashboards, accounting systems, or relayer fee reconciliation that depends on event logs.

Affected Code

- MultiHopRouter.sol#L186C9-L196C11
- MultiHopRouter.sol#L395

Impacts
All MultiHopSwap events emitted on-chain contain totalFees = 0, rendering fee tracking through event logs completely inaccurate. Any off-chain system analytics dashboard, fee revenue reporting, relayer reimbursement calculation, or protocol accounting that relies on the totalFees field from MultiHopSwap events will report zero fees collected for every swap, regardless of actual fees charged.

Remediation

It is suggested to Accumulate fee amounts from every hop's feeDetails into totalFees inside the loop.

Retest

Removed the always-zero totalFees return value from _executeSwap() and removed it from the MultiHopSwap event, eliminating misleading zero-fee reporting

Bug ID #L002 [Acknowledged]

Sequential Nonce Scheme Permanently Blocks Pending Intents When Earlier Nonce Becomes Unexecutable

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Low

Description
The MultiHopRouter contract uses a strictly sequential nonce scheme for both swap intents and join intents. Each user has a monotonically increasing counter (swapNonces[user] and joinNonces[user]), and every intent must carry a nonce that exactly matches the user's current counter value. When an intent executes successfully, the nonce is incremented. When an intent fails and is caught by the try/catch wrapper in executeSwapIntents or executeJoinIntents, the revert rolls back the nonce increment, leaving the counter unchanged. This means the same nonce must be retried in a subsequent batch before any higher nonce can execute.

The problem arises when an intent becomes permanently unexecutable. A user signs intent with nonce 5, but the intent's deadline passes before a relayer submits it, or the user revokes their token approval, or market conditions make the minAmountOut impossible to satisfy regardless of when it is submitted. In any of these scenarios, nonce 5 can never be successfully consumed. Since the contract requires intent.nonce == swapNonces[intent.user], and the nonce counter remains stuck at 5, all subsequent intents signed with nonces 6, 7, 8, and beyond are permanently unreachable. They will always revert with InvalidNonce because the contract will forever be waiting for nonce 5 to succeed first.

Affected Code

- MultiHopRouter.sol#L516C5-L538C35

Impacts
A single expired or permanently failing intent blocks all future intent based operations for that user on the affected nonce track (swap or join). The user must either sign a new valid intent with the exact stuck nonce value and have it successfully execute.

Remediation

It is suggested to replace the sequential nonce scheme with a bitmap based nonce system where each nonce is tracked independently as used or unused, allowing out of order execution.

Retest

This seems like an expected behaviour. Even on using bitmap, user can submit the intent with same nonce id as nonce will only get updated once tx is executed.

Bug ID #L003 [Acknowledged]

setSessionConfig Allows emaAlpha == WAD Which Disables Trade Notional Decay Entirely

Vulnerability Type
Lack of Input Validation (SC04-Lack Of Input Validation)

Severity
Low

Description
In TradeSlicingProtection.setSessionConfig(), the validation on emaAlpha permits WAD (1e18) as a valid value:

```
if (config.emaAlpha > WAD || config.emaAlpha == 0) revert InvalidConfiguration();
// emaAlpha == WAD (1e18) passes validation
```

When emaAlpha == WAD, the _applyEmaDecay function produces no decay regardless of how many blocks have elapsed:

```
uint256 decayFactor = _power(emaAlpha, blocksElapsed); // _power(1e18, n) = 1e18
return (cumulativeNotional * decayFactor) / WAD;    // cumulativeNotional × 1 = unchanged
```

Since 1e18 ^ n = 1e18 for any n, the decayFactor is always WAD, and the cumulative notional never decays. This means a user's session accumulates trade notional indefinitely within the session window, causing sessionized WBF and slippage fees to grow monotonically without decay making trade slicing protection excessively punitive.

Affected Code

- TradeSlicingProtection.sol#L133

Impacts
If the owner sets emaAlpha to WAD (whether intentionally or by misconfiguration), the EMA decay mechanism is completely disabled. The cumulative trade notional within a session never decays between blocks, causing sessionized fees to accumulate without any forgiveness, making the trade slicing protection excessively punitive for all traders on that pool.

Remediation
Disallow emaAlpha == WAD in the validation.

Retest

Clients comment:- acknowledged.

Bug ID #L004 [Fixed]

No Batch Size Limit on Intent Execution Functions

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Low

Description
The executeSwapIntents and executeJoinIntents functions in MultiHopRouter accept an unbounded signedIntents calldata array and iterate over every element without enforcing a maximum batch size. The contract validates that the array is non empty (signedIntents.length == 0 reverts with InvalidAmount) but places no upper bound on the number of intents that can be submitted in a single transaction. Each intent involves computationally expensive operations including ECDSA signature recovery, EIP-712 hash computation, nonce validation, token transfers via safeTransferFrom, multi hop swap execution across one or more pools, fee collection, and event emission. The cumulative gas cost scales linearly with the number of intents in the batch.

Affected Code

- MultiHopRouter.sol#L441
- MultiHopRouter.sol#L481

Impacts
A whitelisted relayer submitting an excessively large batch could create a transaction that consumes the entire block gas limit, potentially causing the transaction to revert and wasting the relayer's gas fees.

Remediation
Introduce a MAX_INTENTS_PER_BATCH constant and validate signedIntents.length <= MAX_INTENTS_PER_BATCH at the top of both executeSwapIntents and executeJoinIntents.

Retest
Added maxIntentsPerBatch (default 50, configurable up to 500) to limit batch size in executeSwapIntents and executeJoinIntents

Bug ID #L005 [Fixed]

Fee Rounding Loss From Integer Division by Number of Hops

Vulnerability Type
Rounding Error

Severity
Low

Description
In WeightedPool._executeSwapExactIn and _executeSwapExactOut, the pool's fee parameters are divided by numHops using standard Solidity integer division to distribute the total fee proportionally across each hop in a multi hop swap. Integer division in Solidity truncates the fractional remainder, meaning the per hop fee rate is slightly lower than the mathematically exact value. When this truncated rate is applied independently at each hop, the cumulative fee collected across all hops is less than the intended total fee by up to numHops - 1 wei in WAD representation.

Affected Code

- WeightedPool.sol#L514C9-L519C10

Impacts
The protocol collects marginally less fees than configured on every multi hop swap.

Remediation
It is suggested to apply the full fee rate on the first hop and compute the remaining hops' rates by subtracting the first hop's allocation from the total or document the bounded rounding behavior.

Retest
Multi-hop fee division changed from truncating integer division (/= n) to FixedPointMathLib.mulDivUp(fee, 1, n) which rounds up, reducing fee undercollection

Bug ID #L006 [Partially Fixed]

Direct Token Donation Can Push Oracle Weights Below MIN_WEIGHT_WAD Causing CapReached Revert

Vulnerability Type
Denial of Service

Severity
Low

Description
WeightedPoolMath.oracleNormalizedWeightsWithDecimals() computes oracle weights using IERC20(token).balanceOf(pool) as the balance source. It reverts with CapReached() if either weight falls below MIN_WEIGHT_WAD (15%):

```
function getTokenBalance(address token, address pool) internal view returns (uint256
balance) {
  return IERC20(token).balanceOf(pool);
}
```

```
if (wA <= MIN_WEIGHT_WAD || wB <= MIN_WEIGHT_WAD) {
  revert CapReached();
}
```

Since balanceOf reflects direct token transfers, an attacker can donate tokens to the pool to inflate one side's balance and push the other side's oracle weight below 15%. This causes all SmartShield-enabled operations (swaps, single-asset joins) to revert, as they all call oracleNormalizedWeightsWithDecimals.

Example:

- Pool is naturally imbalanced with WETH oracle weight at ~16% (just above the 15% min)
- Attacker donates a small amount of USDC directly via IERC20(USDC).transfer(pool, amount) to push WETH weight below 15%
- All SmartShield swaps and single-asset joins revert with CapReached()

- Swaps that would fix the imbalance are also blocked since the weight check occurs before the swap calculation
- Attacker can repeat with small donations to maintain the DoS

Additionally, the MIN_WEIGHT_WAD check is enforced on pre-trade weights rather than post-trade weights. This means corrective swaps that would improve the imbalance are blocked by the current pool state, even though the resulting post-trade weights would be valid.

Affected Code

- WeightedPoolMath.sol#L398

Impacts

An attacker can temporarily DoS all SmartShield operations on a pool by donating tokens to push oracle weights below the minimum threshold. The attack is especially cheap on pools already near the 15% weight boundary, and creates a trap where corrective swaps are also blocked by the same weight check. Note that if SmartShield is disabled, swaps fall into the basic swap path which does not perform the MIN_WEIGHT_WAD check but disabling SmartShield removes all SSP protection (WBF, slippage, trade slicing), which is an unacceptable trade-off.

Remediation

Use internal balance accounting instead of balanceOf for oracle weight calculations to prevent external token donations from affecting the weight computation.

Retest

weightFees() now permits corrective swaps when the pool is already past cap (initMin <= MIN_WEIGHT_WAD && finMin > initMin), converting the original permanent DoS into a self-recoverable state. Non-improving trades are correctly blocked during imbalance. Same logic added in SingleJoinMath. Root cause (getTokenBalance still uses balanceOf(pool)) remains, so donation-triggered cap breaches are still possible, but the economic cost to the attacker (donated tokens are lost) and arbitrage incentives for quick recovery significantly reduce practical impact.

Bug ID #L007 [Fixed]

Capped WBF Not Reflected in sessionResult Causing Inflated Session State

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
In both OutGivenInMath and InGivenOutMath, after the sessionized WBF is retrieved from calculateSessionizedFees, it is capped in two places:

```
// Cap 1: sessionizedWbfAmount capped to amountIn
if (sessionizedWbfAmount > amountIn) {
  sessionizedWbfAmount = amountIn;
}

// Cap 2: wbfPercentage capped to MAX_WBF_WAD (99%)
if (wbfPercentage > WeightedPoolMath.MAX_WBF_WAD) {
  wbfPercentage = WeightedPoolMath.MAX_WBF_WAD;
  sessionizedWbfAmount = (WeightedPoolMath.MAX_WBF_WAD * amountIn) / WAD;
}
```

The local sessionizedWbfAmount and wbfPercentage are correctly capped and used for the swap output calculation. However, sessionResult.sessionizedWbf (the original uncapped USD value) is never updated to reflect the cap. This uncapped value is then passed to SSPCore.updateSessionState:

```
tradeSlicingProtection.updateSessionState(
  ...,
  sessionResult.sessionizedWbf,  // <<<< uncapped value
  ...
);
```

The session records a higher WBF than was actually charged to the user.

Affected Code

- OutGivenInMath.sol#L246C9-L257C14
- InGivenOutMath.sol#L244C12-L255C14

Impacts

The session's cumulativeWbf accumulates more WBF than was actually charged, creating a mismatch between recorded and actual fees. While cumulativeWbf is currently not used to influence subsequent fee calculations (making the immediate impact limited), it produces inaccurate session accounting and could cause issues if cumulativeWbf is ever used for fee logic or reporting in future updates.

Remediation

Update sessionResult.sessionizedWbf after capping to reflect the actual WBF amount charged, so that SSPCore.updateSessionState receives the accurate capped value.

Retest

sessionResult.sessionizedWbf is now updated after capping in both OutGivenInMath and InGivenOutMath, so updateSessionState receives the actual charged WBF value

Bug ID #L008 [Not Fixed]

weightFees Charges WBF on Neutral Trades Where diff == 0

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
In WeightedPoolMath.weightFees(), the branching logic for determining improving vs worsening trades uses:

```
if (flipped) {
  // Worsening WBF charged
} else if (diff < 0) {
  // Improving WBR given, fee suppression applied
} else {
  // diff >= 0 WBF charged
}
```

The else branch catches both diff > 0 (worsening) and diff == 0 (neutral no change in distance to target). When diff == 0, the trade has not moved pool weights closer to or further from the target, so it should be treated as neutral with no WBF or WBR applied. Instead, the trade is treated as worsening and WBF is charged.

This can occur when a trade's size and price impact result in identical pre-trade and post-trade distances from target weights.

Affected Code

- WeightedPoolMath.sol#L517

Impacts
When a trade results in zero net change to the pool's distance from target weights (diff == 0), the user is penalized with a Weight Breaking Fee despite having no adverse effect on pool balance. This creates an unfair cost for users whose trades are weight-neutral, and since WBF is deducted from the swap output, it directly reduces the amountOut the user receives. While this edge case requires a precise trade size that results in identical pre- and post-trade distances, it is possible and the protocol should not penalize users for trades that have no weight impact.

Remediation

Add an explicit check for the diff == 0 case before the worsening branch. When the trade is neutral, neither WBF nor WBR should apply. Split the current else branch to handle the neutral and worsening cases separately:

Retest

diff == 0 still falls into the else (worsening) branch. No explicit neutral-trade handling added.

Bug ID #L009 [Acknowledged]

WBR Distribution Silently Capped When Treasury Has Insufficient Funds

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
In PoolOperations._processWrrDistribution(), the WBR reward is silently capped to the treasury's available balance:

```
uint256 treasuryBalance = IERC20(wbrFee.token).balanceOf(treasury);
actualAmountDistributed = wbrFee.amount < treasuryBalance
  ? wbrFee.amount
  : treasuryBalance;
```

If the treasury doesn't hold enough tokens, the user receives a partial WBR reward with no revert or event indicating the shortfall. The swap's fee suppression (reduced swap/taker fees for improving trades) has already been applied based on the full WBR amount, but the actual reward paid out is less.

Additionally, the function checks the treasury's balance but not its allowance to the pool. If the treasury has not approved the pool for sufficient allowance, the safeTransferFrom reverts. This call is in the main swap flow without try/catch, so the revert propagates and blocks the entire swap.

Affected Code

- PoolOperations.sol#L217C4-L227C6

Impacts
Users who perform improving trades receive less WBR reward than expected when the treasury is underfunded. Since WBR is a bonus incentive and fee suppression is already applied independently, the direct financial impact is limited to the missed reward amount.

Remediation
Record the unpaid WBR shortfall in a pending rewards mapping (e.g., mapping(address => uint256) pendingWbr) so users can claim their owed reward later when the treasury is replenished.

Retest

Client's Comment:- this is the expected behaviour and don't want user claim left rewards.

Bug ID #L010 [Acknowledged]

Trade Slicing Protection Bypassed When SmartShield Is Disabled Despite Independent Config

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
TradeSlicingProtection has its own independent per-pool configuration with a separate enabled flag:

```
struct SessionConfig {
  uint256 emaAlpha;
  uint32 tMax;
  uint32 bMax;
  uint32 cleanupTime;
  bool enabled;   // independent enable/disable
}
```

This config is set by the TradeSlicingProtection owner via setSessionConfig(), completely independent of SmartShield. However, in WeightedPool.executeSwapExactIn and _executeSwapExactOut, trade slicing is only invoked inside the SmartShield path:

```
if (cachedParams.smartShieldEnabled) {
  // CEX price verification + SSPCore (which includes trade slicing)
  (amountOut, feeDetails) = sspCore.handleSSPoolSwap(
    ..., tradeSlicingProtection, ...
  );
} else {
  // Basic path NO trade slicing at all
  (amountOut, feeDetails) = quoteOutGivenInBasic(...);
}
```

When smartShieldEnabled = false, the basic swap path is used which does not invoke tradeSlicingProtection at all even if SessionConfig.enabled = true for that pool. Since every pool is deployed with an oracle that provides token prices, the data needed for trade slicing (trade

notional, exec price, start price) is available regardless of SmartShield status. The two features are coupled in the code when they should be independent.

Affected Code

- WeightedPool.sol#L542
- WeightedPool.sol#L631

Impacts

When SmartShield is disabled (e.g., temporarily for maintenance or by pool owner choice), trade slicing protection is silently bypassed even though it has been explicitly configured and enabled for the pool. Traders can freely slice large trades into smaller ones without any session tracking or cumulative fee enforcement, defeating the anti-slicing mechanism.

Remediation

Invoke trade slicing protection in the basic swap path as well when the pool's SessionConfig.enabled is true.

Retest

This issue is acknowledged.

Bug ID #L011 [Acknowledged]

Cumulative Value Regression Permanently Locks User Claims

Vulnerability Type
State Integrity Risk

Severity
Low

Description
The contract assumes cumulative earnings are monotonically increasing across epochs but does not enforce this on-chain. If a submitted Merkle root contains a cumulative value lower than what a user has already claimed, `claim()` reverts with `CumulativeTooLow()`. Since `totalClaimed[user]` only ever increases and claims validate exclusively against `currentEpoch`, the user has no fallback to a previous valid root.

The contract stores no per-user cumulative history from prior epochs, making it impossible to detect or prevent regression on-chain. The affected user remains permanently locked until governance submits a corrected root in a future epoch.

Affected Code

- RevenueDistributor.sol#L176-L180

Impacts
An incorrect or malicious epoch submission can permanently lock specific users from claiming rewards. The issue does not allow unauthorized fund extraction but can cause irreversible claim denial without governance intervention.

Remediation
Ensure strict off-chain validation to guarantee monotonic cumulative values before submission, or introduce a mechanism that prevents regression of cumulative totals across epochs.

Retest
This issue has been acknowledged.

Bug ID #L012 [Fixed]

Out-of-Order Epoch Submission Silently Accepts Unreachable Root

Vulnerability Type
Lack of Input Validation (SC04-Lack Of Input Validation)

Severity
Low

Description
The contract accepts epoch submissions regardless of whether the submitted epoch is below currentEpoch. When a lower epoch is submitted after a higher one, its Merkle root is stored and USDC is deposited, but since claim() exclusively reads currentEpoch, that root can never be reached by any user.
The contract silently accepts the deposit without any warning or rejection. The deposited USDC is not immediately stranded future cumulative roots from subsequent epochs can absorb the liability but if the bot never includes that epoch's reward data in a future root, the funds become permanently unclaimable without governance intervention.

Affected Code

- RevenueDistributor.sol#L149-L152
- RevenueDistributor.sol#L168-L171

Impacts
USDC is deposited against a Merkle root that claim() can never access. If the bot fails to carry forward affected reward data into a future epoch, those funds accumulate in the contract with no user-facing claim path and require manual governance recovery.

Remediation
Reject submissions where the epoch is below currentEpoch to prevent USDC being deposited against an unreachable root

Retest
This issue has been fixed.

Bug ID #L013 [Fixed]

claim( ) Hardcodes currentEpoch Proof Race Condition

Vulnerability Type
Design Weakness

Severity
Low

Description
The claim() function hardcodes verification against currentEpoch instead of accepting an explicit epoch parameter from the caller. Users generate Merkle proofs off-chain for a specific epoch, but if a new epoch is submitted by the bot before the user's claim transaction is mined, currentEpoch advances and the proof becomes invalid against the new epoch's root. The transaction then reverts with InvalidProof, burning the user's gas. This creates a race condition between bot epoch submissions and user claim transactions.

Affected Code

- RevenueDistributor.sol#L169-L172

Impacts
Repeated claim failures during high-frequency epoch updates leading to gas loss and UX degradation. No accounting corruption, no double-claim vulnerability, and no privilege escalation. Increased support burden and potential frontend confusion during active epoch transitions.

Remediation
Allow an explicit epoch parameter in the claim( ) function and verify the proof against the supplied epoch root

Retest
This issue has been fixed.

Bug ID #L014 [Acknowledged]

Role Event Bypass Causing Off-Chain Desync

Vulnerability Type
Event Inconsistency

Severity
Low

Description
The contract provides custom wrapper functions addBotOperator() and removeBotOperator() that emit BotOperatorAdded and BotOperatorRemoved events. However, since the contract inherits OpenZeppelin's AccessControl, governance holding DEFAULT_ADMIN_ROLE can directly call the inherited grantRole() and revokeRole() functions to manage BOT_OPERATOR_ROLE.

These inherited functions only emit the standard RoleGranted/RoleRevoked events, bypassing the custom events entirely. Additionally, direct grantRole() calls bypass the _validateAddress zero-address check enforced by the custom wrappers. If off-chain monitoring systems rely solely on custom events to track operator state, they will desync from actual on-chain role assignments.

Affected Code

- RevenueDistributor.sol#L271-L286
- MerkleDistributor.sol#L218-L234

Impacts
Off-chain infrastructure relying on custom events will miss role changes made via direct grantRole()/revokeRole() calls. DAO interfaces commonly invoke inherited role functions directly, making event mismatch likely in production. On-chain logic remains correct, but off-chain operator tracking, dashboards, and alerting systems may break silently.

Remediation
Override _grantRole and _revokeRole to emit the custom events when BOT_OPERATOR_ROLE is involved, ensuring consistent event emission regardless of the call path.

Retest
This issue has been acknowledged.

Bug ID #L015 [Fixed]

Global Reward Accounting Undercount

Vulnerability Type
Accounting Inconsistency

Severity
Low

Description
The contract exposes two distinct reward payout paths. The first path is through the explicit claimRewards() function, which transfers pending rewards for a single specified token and increments totalClaimsByToken[token] += claimedAmount at the end of the call. The second path is through the implicit harvest that occurs inside _harvestAndSyncRewardDebt(), which is invoked by both _stake() and unstake() on every staking or unstaking operation.

This implicit harvest iterates over all whitelisted reward tokens, computes each token's pending amount via CumulativeRewardIndex.calculatePendingRewards(), and transfers any non-zero pending amount to the user via _safeRewardTransfer().

However, _harvestAndSyncRewardDebt() never updates totalClaimsByToken for any of the tokens it pays out. As a result, every reward payout that occurs through the implicit harvest path during stake() or unstake() is excluded from the global aggregate tracking.

The totalClaimsByToken mapping only reflects rewards paid through explicit claimRewards() calls. Since users frequently trigger implicit harvests simply by adjusting their LP positions, the cumulative undercount will grow over the lifetime of the vault, producing an increasingly inaccurate aggregate figure. This does not affect per-user reward correctness because rewardDebt and pendingClaim are updated correctly in both paths.

Affected Code

- LPGaugeVault.sol#L265
- LPGaugeVault.sol#L185-L203

Impacts
totalClaimsByToken will consistently underreport actual rewards distributed because payouts executed via _harvestAndSyncRewardDebt() are not reflected in this counter. This creates inaccurate global accounting and may mislead monitoring systems or solvency checks that rely on

this metric, although individual user reward calculations remain correct and no direct fund loss occurs.

Remediation

Increment `totalClaimsByToken[token]` inside `_harvestAndSyncRewardDebt()` after calculating `amountReceived` so all reward transfers are consistently recorded in the global accounting state.

Retest

This issue has been fixed as `totalClaimsByToken[token] += amountReceived` is now executed in the stake path harvest.

Bug ID #L016 [Acknowledged]

Permissionless Reward Claim Enables Forced Harvest

Vulnerability Type
Missing Access Control

Severity
Low

Description
The claimRewards(address user, address token) function allows any external caller to trigger a reward claim for any staker without verifying that msg.sender matches the specified user or is an authorized delegate.

Although rewards are transferred to the correct user address, the function updates the user's rewardDebt and settles the accounting state as part of the claim process.

This permits third parties to forcibly harvest rewards on behalf of users, modifying their reward accounting state without consent. While no funds are stolen, users lose control over claim timing and reward settlement behavior.

Affected Code

- LPGaugeVault.sol#L240-L269

Impacts
Any external actor can force reward harvesting for arbitrary users, removing their ability to control claim timing. Although funds are transferred to the rightful owner and no direct financial loss occurs, this introduces a griefing vector and may interfere with user strategies or future protocol features such as claim cooldowns, reward locking, or delayed-claim incentives.

Remediation
Restrict claimRewards() to msg.sender == user, or implement an explicit delegation mechanism (e.g., approved claimer mapping) so only authorized addresses can trigger reward claims on behalf of a user.

Retest
This issue has been acknowledged.

Bug ID #L017 [Fixed]

Raw ecrecover Usage Without Signature Malleability Protection in verifyCexPrice

Vulnerability Type
Cryptographic Weakness

Severity
Low

Description
The SSPCore.verifyCexPrice function recovers the signer of CEX price data using Solidity's built-in ecrecover precompile directly, without applying any signature malleability protection. The function performs a zero address check (if (recoveredSigner == address(0)) revert InvalidSignature()) but does not validate that the s value falls in the lower half of the curve order, which is the standard mitigation defined in EIP-2 and implemented by OpenZeppelin's ECDSA.recover.

Affected Code

- SSPCore.sol#L251

Impacts
An attacker who observes a valid CEX price signature in a pending transaction's calldata can compute the mathematically equivalent malleable signature and submit their own transaction using the modified (v, r, s) values. Both signatures recover to the same trusted signer, so the isTrustedSigner check passes in either case.

Remediation
Replace the raw ecrecover call with OpenZeppelin's ECDSA.recover function.

Retest
Raw ecrecover in SSPCore.verifyCexPrice replaced with OpenZeppelin's ECDSA.recover(), adding EIP-2 signature malleability protection

Bug ID #L018 [Fixed]

New Liquidity Bands Default to 0% Minimum Slippage Floor

Vulnerability Type
Logic Error

Severity
Low

Description
When an authorized feeder updates liquidity bands via WeightedPoolFactory.feedMultipleExternalLiquidityRatios(), it provides LiquidityBandConfig structs that only contain thresholdWad and ratioWad the minSlippageWad field is not part of this struct. The AdminOperations.updateLiquidityBands function preserves minSlippageWad from existing bands by index, but any newly added band (where the new index exceeds oldBandCount) defaults to minSlippageWad = 0.

Since bands are ordered by ascending trade-size thresholds, newly added bands correspond to the largest trade sizes. These large-trade bands also carry the highest ratioWad values (e.g., 200x–500x), which divide the trade down to a tiny resized amount before passing it to the Balancer curve. A tiny resized amount produces near-zero natural curve slippage. The minSlippageWad floor exists precisely to guarantee a minimum cost for such trades without it, a whale trade can execute at near-oracle spot price with effectively zero slippage.

The pool owner can fix this by calling updateFeesAndMinSlippage(), but this is a separate transaction by a different role (pool owner vs. authorized feeder). Between the feeder's band update and the owner's fix, there is a vulnerable window during which arbitrageurs can exploit the zero-slippage bands.

Affected Code

- AdminOperations.sol#L63

Impacts
Large trades hitting newly added bands execute with near-zero slippage, allowing arbitrageurs to trade at near-oracle prices at the expense of liquidity providers until the pool owner separately patches minSlippageWad via updateFeesAndMinSlippage().

Remediation

Include minSlippageWad in the LiquidityBandConfig struct so the feeder sets it atomically, or default new bands to the last existing band's minSlippageWad instead of zero.

Retest

New liquidity bands now default to MINIMUM_SLIPPAGE = 5e14 (0.05%) instead of 0% in AdminOpe

Bug ID #L019 [Fixed]

Incorrect Weight Parameter in WBF Calculation Uses Math.min Instead of Math.max

Vulnerability Type
Logic Error

Severity
Low

Description
In WeightedPoolMath.weightFees, the Weight Breaking Fee (WBF) for worsening and flipped trades is computed using Math.min(finIn, finOut) the lighter asset's weight after the swap. This value is passed to computeWBFCharge as the w parameter, which internally computes the skew penalty via F_pow(w) ∝ (w / (1 - w))^2.5.

Using the lighter side's weight produces an inverted relationship: the more imbalanced the pool becomes after a trade, the lower w is, and thus the lower the WBF penalty. A trade that worsens the pool from 55/45 to 80/20 pays less WBF than a trade that worsens it from 55/45 to 60/40, which is the opposite of the intended incentive.

Additionally, lambdaBlend(w) activates the F_pow skew penalty only when w > 0.65e18. Since min(finIn, finOut) for a 2-asset pool is always ≤ 0.5 (and strictly less than 0.5 when imbalanced), lambdaBlend always returns 0 for the min weight, making the skew penalty component effectively dead code in worsening/flipped trade paths.

By contrast, the improving trade path (line 597) correctly uses Math.max(initIn, initOut) the heavy side which activates lambdaBlend and produces higher rewards for fixing larger imbalances.

Affected Code

- [WeightedPoolMath.sol#L574]

Impacts
Trades that worsen pool balance pay a negligible WBF because the skew penalty (F_pow) never activates — min weight is always ≤ 0.5 and below lambdaBlend's 0.65 activation threshold. This removes the primary economic disincentive against imbalancing the pool, allowing arbitrageurs to push pool weights to extremes with minimal penalty.

Remediation

Replace Math.min(finIn, finOut) with Math.max(finIn, finOut) in both the flipped and worsening branches so that the WBF scales with the severity of the imbalance, consistent with the improving path's use of Math.max.

Retest

WBF calculation in weightFees now uses oracleWeightHeavyWad (Math.max(finIn, finOut)) instead of Math.min(finIn, finOut), properly activating the skew penalty for imbalanced pools

Bug ID #L020 [Not Fixed]

Missing Handling of Fees on Transfer in ERC20 Token Transfers

Vulnerability Type
Calculation Inaccuracy

Severity
Low

Description
The contract contains a vulnerability related to transferring ERC20 tokens without considering the possibility of fees charged on transfer. Some ERC20 tokens implement a fee mechanism, where a certain percentage of tokens is deducted as a fee during each transfer. However, the contract does not account for this possibility when transferring tokens using the safeTransferFrom/TransferFrom function.

Affected Code

- MultiHopRouter.sol#L166C5-L199C6

Impacts
Failure to account for transfer fees can lead to accounting errors and financial losses. When tokens with transfer fees are transferred, the actual received amount may be less than expected due to the deduction of fees. As a result, the contract's internal accounting and balance tracking may become inaccurate, leading to discrepancies in token balances and potential financial losses for users.

Remediation
To address this vulnerability it is recommended to add a mechanism to calculate the balance of the contract before and after the transfer is completed.

Retest
This issue is not fixed.

Bug ID #L021 [Fixed]

Epoch Advancement Cap May Record Activity in Stale Epoch After Long Dormancy

Vulnerability Type
Temporal State Inconsistency

Severity
Low

Description
The _updateEpoch() function limits epoch progression to MAX_EPOCHS_PER_CALL = 100 to prevent excessive gas usage. If the contract remains inactive for more than 100 epochs, a single state-changing call will only advance 100 epochs, even if more time has elapsed.

As a result, the first interaction after prolonged dormancy may record activity in an intermediate epoch rather than the true current calendar epoch. The system can be fully synchronized through repeated calls to advanceEpochs(), which is permissionless.

Affected Code

- GaugesManager.sol#L713-L736
- GaugesManager.sol#L660-L671

Impacts
Following extended inactivity, activity may temporarily be recorded in a stale epoch until additional epoch-advance calls are executed. This could slightly distort emission accounting for that specific epoch, though the condition requires prolonged dormancy and self-corrects once epochs are fully advanced.

Remediation
Document the need for periodic advanceEpochs() calls via a keeper mechanism, or redesign epoch advancement to compute skipped epochs arithmetically without iterative limits, ensuring full synchronization in a single call while remaining gas safe.

Retest
This issue has been fixed by removing the vulnerable file.

Bug ID #L022 [Fixed]

Per-Wallet Emission Cap Excess Is Not Redistributed

Vulnerability Type
Business Logic Flaw

Severity
Low

Description
When a user's calculated emission exceeds perWalletCap, the excess amount is truncated and permanently discarded rather than redistributed or rolled forward.

```
if (perWalletCap > 0 && emissionAmount > perWalletCap) {
   emissionAmount = perWalletCap;
}
```

As a result, total distributed emissions for an epoch can be lower than the configured emission amount. While this may be intentional to limit concentration, it leads to unused emissions.

Affected Code

- GaugesManager.sol#L778-L828

Impacts
If users hit the cap, part of the emission allocation for that epoch is permanently lost, reducing overall distribution efficiency without reallocating the excess to other participants.

Remediation
If full emission utilization is required, track capped excess per epoch and either redistribute it among eligible users or roll it forward into the next epoch's emission pool.

Retest
This issue has been fixed by removing the vulnerable file.

Bug ID #L023 [Fixed]

Slippage Protection Bypass via Unbacked Weight Breaking Reward Payouts

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
The _executeSwapExactIn function in WeightedPool performs a slippage check that includes the theoretical Weight Breaking Reward (WBR) amount as part of the user's expected output. The check computes amountOut + feeDetails.wbrFee.amount and compares it against the caller-specified minAmountOut, treating the WBR payout as a guaranteed component of the total output. However, the actual WBR distribution, which occurs later during _collectAndDistributeFees, is silently capped to the treasury's available token balance inside PoolOperations._processWrrDistribution. This function reads the treasury's balanceOf and takes the minimum of the theoretical WBR amount and the treasury balance, meaning the actual transferred reward can be significantly lower than the value used in the slippage check. The feeDetails.wbrFee.amount field is updated to reflect the capped value after distribution, but this update occurs after the slippage check has already passed, rendering the correction ineffective for user protection. The same pattern exists in _executeSwapExactOut, where feeDetails.wbrFee.amount is similarly included in the slippage validation and later capped during fee distribution.

Affected Code

- WeightedPool.sol#L546

Impacts
Users who set minAmountOut based on an expectation that both the AMM output and the WBR reward will be delivered can receive a total amount below their specified minimum without the transaction reverting.

Remediation
Move the slippage check to after the fee distribution so it can use the actual capped WBR amount, or perform the treasury balance check and WBR capping before the slippage validation.

Retest

This issue is fixed by moving slippage check after WBR is capped in amount.

Bug ID #L024 [Fixed]

setEpochLength Missing Same-Value Validation

Vulnerability Type
Improper Input Validation

Severity
Low

Description
The setEpochLength function does not verify whether newEpochLength is equal to the current epochLength. As a result, an admin can set the same value again, causing an unnecessary state write and emitting an EpochLengthUpdated event despite no actual configuration change.

This creates avoidable gas usage and produces misleading off-chain logs suggesting that a parameter update occurred when it did not.

Affected Code

- GaugesManager.sol#L647-L660

Impacts
Calling the function with the same value results in wasted gas and unnecessary event emission, potentially confusing monitoring systems or off-chain analytics that rely on event logs to track parameter changes. There is no financial or protocol integrity impact.

Remediation
Add a same-value check to prevent redundant updates:

```
if (newEpochLength == epochLength) revert InvalidEmissionParams();
```

Retest
This issue has been fixed by removing the vulnerable file.

Bug ID #L025 [Fixed]

Emission Threshold Creates Dead Zone for Low-Activity Categories

Vulnerability Type
Business Logic Flaw

Severity
Low

Description
The calculateClassEmission function returns zero emissions when a category's total activity is below minThreshold.

```
if (activity < minThreshold) {
   return 0;
}
```

This introduces a cold-start issue for categories with naturally low early participation. Since emissions are suppressed below the threshold, users may lack incentive to contribute activity, which in turn prevents the category from ever surpassing the threshold organically.

Although minThreshold is configurable by the admin, the mechanism applies uniformly across all categories.

Affected Code

- EmissionControl.sol#L64-L91

Impacts
Categories with low initial activity may receive zero emissions, slowing adoption and requiring manual parameter adjustments by governance to enable bootstrapping. No funds are lost, but incentive alignment may be weakened during early growth phases.

Remediation
Consider implementing per-category thresholds instead of a single global minThreshold, or introduce a bootstrapping configuration phase with reduced or dynamic thresholds to support early-stage growth.

Retest

This issue has been fixed by removing the vulnerable file.

Bug ID #L026 [Fixed]

Potential Double Weight Application in removeActivity

Vulnerability Type
Integration Risk

Severity
Low

Description
The removeActivity function internally applies _applyPoolWeight to the provided amount. If an integrating contract mistakenly pre-adjusts the value using the pool weight before calling removeActivity, the weight will be applied twice.

This results in a smaller-than-intended adjusted amount being removed from the user's recorded activity. While internal usage is symmetrical and correct when raw values are supplied, the function design creates an integration footgun if external callers misunderstand the expected input format.

Affected Code

- GaugesManager.sol#L197-L213
- GaugesManager.sol#L828-L846

Impacts
If pre-weighted values are passed, activity will be under-removed, leaving residual activity that can inflate the user's emission share. The issue only arises from incorrect integration and does not affect correct protocol usage.

Remediation
Clearly document that removeActivity expects raw, unweighted amounts. Optionally, provide a dedicated function that accepts pre-weighted values to prevent accidental double application of the pool weight.

Retest
This issue has been fixed by removing the vulnerable file.

Bug ID #L027 [Fixed]

totalStandardUnits Rounding Drift From Per-User Decay Settlements

Vulnerability Type
Accounting Precision Issue

Severity
Low

Description

In _settleDecayForCategory, decay is calculated per user using floor division. Because rounding is applied individually, the sum of per-user decayedAmount values can slightly exceed the mathematically exact decay of the aggregate total.

totalStandardUnits is reduced by the accumulated per-user decay amounts. Due to rounding drift, this can cause the global tracker to decrease marginally more than intended over time. If decayedAmount exceeds totalStandardUnits, the contract silently sets the tracker to zero instead of reverting.

Although each rounding discrepancy is bounded (at most 1 wei per settlement), repeated settlements can accumulate measurable drift over a long period.

Affected Code

- UnitsRegistry.sol#L657-L713
- UnitsRegistry.sol#L587

Impacts
Over time, totalStandardUnits may drift below the true aggregate of user balances due to rounding discrepancies. In extreme cases, the silent zeroing branch can reset the global tracker entirely, permanently desynchronizing it from actual balances and potentially affecting any logic that depends on this global value.

Remediation
Avoid silent zeroing by reverting or explicitly emitting a drift event when inconsistency is detected.

Retest

Bug ID #L028 [Fixed]

Protocol Fees Incorrectly Calculated on Slippage-Adjusted Amounts in SmartShield Swap Paths

Vulnerability Type
Accounting Error

Severity
Low

Description
In both SmartShield swap paths, protocol fees (swap fee, taker fee, WBF) are applied to slippage-adjusted amounts instead of the oracle-derived fair value. Slippage is a natural AMM curve cost and should be independent of protocol fees, but the current implementation entangles them.

In OutGivenInMath, slippage is deducted from oracleOut first, then fees are applied to the reduced amount. This shrinks the fee base, causing the protocol and LPs to collect less in fees than intended.

In InGivenOutMath, slippage is added to oracleIn first, then fees are applied to the inflated amount. This causes users to be charged fees on the slippage portion they already bear.

Affected Code

- InGivenOutMath.sol#L260C7-L263C1
- OutGivenInMath.sol#L262C1-L265C1

Impacts
In the ExactIn path, the protocol and LPs collect less fees than intended because slippage reduces the fee base. In the ExactOut path, users are overcharged because fees are applied to a slippage-inflated input. Both effects are small at typical fee and slippage levels but create an asymmetry where identical trades produce different effective fee rates depending on swap direction.

Remediation
Convert slippage to a percentage and combine it additively with other fee percentages so that all deductions are independent of each other.

Retest

Fees now applied as additive percentages to oracle value (oracleOut * (1 - totalFeeWad) where totalFeeWad = slippage% + swapFee + takerFee + wbf%), making fees independent of each other.

Bug ID #L029 [Fixed]

Missing Zero-Address Check in transferGovernance Can Permanently Lock Governance

Vulnerability Type
Input Validation

Severity
Low

Description
The RyzeRouter.transferGovernance function does not validate that newGovernor is not address(0). If the governor accidentally passes address(0), governance is permanently renounced with no recovery mechanism. This is inconsistent with the constructor, which explicitly checks for zero addresses before setting the governor.

All onlyGovernor functions — govSetConfig, govToggleOracleSwitch, and transferGovernance itself — become permanently inaccessible, preventing any future oracle configuration changes.

Affected Code

- RyzeRouter.sol#L117

Impacts
An accidental transferGovernance(address(0)) call irreversibly locks all governor-only operations, including oracle configuration, oracle switch toggling, and further governance transfers. The entire oracle routing layer becomes immutable with no way to add, update, or switch oracle providers.

Remediation
Add a zero-address check in transferGovernance consistent with the constructor: require(newGovernor != address(0)). If intentional renouncement is desired, implement a separate renounceGovernance() function to prevent accidental zero-address transfers.

Retest
Added zero-address check in RyzeRouter.transferGovernance: reverts with PriceOracle_InvalidConfiguration if newGovernor == address(0)

Bug ID #L030 [Fixed]

Floating and Outdated Pragma

Vulnerability Type
Floating Pragma (SCWE-060)

Severity
Low

Description
Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.
The contract allowed floating or unlocked pragma to be used, i.e., >= 0.8.23. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code

- RyzeRouter.sol#L2
- IPriceOracle.sol#L2
- IOracle.sol#L2
- BaseAdapter.sol#L2
- PythOracle.sol#L2
- Errors.sol#L2
- ScaleUtils.sol#L2
- TradeSlicingProtection.sol#L2
- WeightBreakingRewardConfig.sol#L2
- PoolCreator.sol#L2
- WeightedPoolQueries.sol#L2
- SSPCore.sol#L2
- WeightedPool.sol#L2
- WeightedPoolFactory.sol#L2
- MultiHopRouter.sol#L2
- IAMM.sol#L2
- IMultiHopRouter.sol#L2

- ISSPCore.sol#L2
- IWeightedPoolQueries.sol#L2
- IWeightBreakingReward.sol#L2
- ITradeSlicingProtection.sol#L2
- SingleJoinMath.sol#L2
- PoolOperations.sol#L2
- OutGivenInMath.sol#L2
- BalancerMath.sol#L2
- InGivenOutMath.sol#L2
- WeightedPoolMath.sol#L2
- AdminOperations.sol#L2
- SwapOperations.sol#L2
- FixedPointMathLib.sol#L2

Impacts

If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.

Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.

The likelihood of exploitation is low.

Remediation

Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.31 pragma version

Reference: https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/

Retest

All contract files pinned from floating ^0.8.23 to fixed 0.8.30 pragma

Bug ID #L031 [Partially Fixed]

Missing events in important functions

Vulnerability Type
Missing Best Practices

Severity
Low

Description
Events are inheritable members of contracts. When you call them, they cause the arguments to be stored in the transaction's log—a special data structure in the blockchain. These logs are associated with the address of the contract which can then be used by developers and auditors to keep track of the transactions.

The contract was found to be missing these events on certain critical functions which would make it difficult or impossible to track these transactions off-chain.

Affected Code

- RyzeRouter.sol#L241
- WeightedPool.sol#L934
- PoolCreator.sol#L14
- WeightedPool.sol#L855
- WeightedPool.sol#L868
- WeightedPool.sol#L893
- WeightedPool.sol#L898
- WeightedPool.sol#L903
- WeightedPool.sol#L908
- WeightedPool.sol#L914
- WeightedPool.sol#L933
- WeightedPool.sol#L994
- WeightedPoolFactory.sol#L175
- WeightedPoolFactory.sol#L182
- MultiHopRouter.sol#L145

Impacts

Events are used to track the transactions off-chain and missing these events on critical functions makes it difficult to audit these logs if they're needed at a later stage.

Remediation

Consider emitting events for important functions to keep track of them.

Retest

This issue is partially fixed as some function still lacks proper event emission.

Bug ID #I001 [Fixed]

Incorrect Error Used for Zero Merkle Root

Vulnerability Type
Incorrect Error

Severity
Informational

Description
In _validateEpochSubmission(), a zero merkle root reverts with InvalidAmount(). A merkle root is not an amount, making the error semantically incorrect. This can cause confusion during debugging and off-chain error handling, as InvalidAmount suggests a numerical input issue rather than an invalid root submission.

Affected Code

- MerkleDistributor.sol#L242-L250

Impacts
Misleading error signaling for off-chain integrations and debugging. No functional or security impact, but reduces code clarity and developer experience.

Remediation
Introduce a dedicated error for invalid merkle root and use it in the validation.

Retest
This issue has been fixed.

Bug ID #I002 [Fixed]

Redundant State Update

Vulnerability Type
Redundant State Update

Severity
Informational

Description
The claimRewards() function updates index.lastUpdateTime and index.totalEligibleUnits when certain conditions are met. However, these fields do not influence reward calculations, as pending rewards are derived solely from accRewardPerShare. Since accRewardPerShare is only updated during distributeRewards(), mutating these timestamp-related fields inside claimRewards() has no functional effect on reward distribution.

As a result, the function performs unnecessary state writes that do not impact protocol behavior and may create confusion by implying active reward index updates are occurring during claims.

Affected Code

- LPGaugeVault.sol#L249-L252

Impacts
There is no financial impact. The redundant SSTORE operations increase gas costs for claim transactions and introduce misleading state changes that may complicate maintenance or auditing.

Remediation
Remove the unnecessary timestamp update block from claimRewards() unless it is required for future emission logic that directly affects accRewardPerShare; otherwise, avoid state mutations that do not influence reward computation.

Retest
This issue has been fixed.

Bug ID #I003 [Fixed]

Missing zero address validations

Vulnerability Type
Missing Input Validation (SC04-Lack Of Input Validation)

Severity
Informational

Description
The contracts were found to be setting new addresses without proper validations for zero addresses.
Address type parameters should include a zero-address check otherwise contract functionality may become inaccessible or tokens burned forever.
Depending on the logic of the contract, this could prove fatal and the users or the contracts could lose their funds, or the ownership of the contract could be lost forever.

Affected Code

- RyzeRouter.sol#L205C5-L208C6
- WeightedPool.sol#L808
- WeightedPoolFactory.sol#L182

Impacts
If address type parameters do not include a zero-address check, contract functionality may become unavailable or tokens may be burned permanently.

Remediation
Add a zero address validation to all the functions where addresses are being set.

Retest
zero-address validations added to constructor and admin setter functions in WeightedPool.sol and WeightedPoolFactory.sol

Bug ID #I004 [Fixed]

Dead Code

Vulnerability Type
Code With No Effects (SCWE-062)

Severity
Informational

Description
It is recommended to keep the production repository clean to prevent confusion and the introduction of vulnerabilities. The functions and parameters, contracts, and interfaces that are never used or called externally or from inside the contracts should be removed when the contract is deployed on the mainnet.

Affected Code

- RyzeRouter.sol#L59

Impacts
This does not impact the security aspect of the Smart contract but prevents confusion when the code is sent to other developers or auditors to understand and implement.
This reduces the overall size of the contracts and also helps in saving gas.

Remediation
If the library functions are not supposed to be used anywhere, consider removing them from the contract.

Retest
Dead code removed: fallbackOracleFeederPrice mappings from RyzeRouter; poolSharesOutGivenSingleAssetIn and feeRatio from BalancerMath; safeMultiply from WeightedPoolMath.

Bug ID #I005 [Not Fixed]

powWadUp And sqrtWadUp Provide No Rounding-Up Guarantee For Fractional Exponents

Vulnerability Type
Informational

Severity
Informational

Description
The FixedPointMathLib library provides powWadDown() and powWadUp() to raise a WAD-scaled base to a WAD-scaled exponent with directional rounding. For integer exponents (1, 2, or 3 WAD), the functions correctly diverge — powWadDown() uses mulDivDown while powWadUp() uses mulDivUp. For all other exponents, both functions fall through to the same PRBMath pow() call and return identical results.

```
// powWadDown — line 59-63
UD60x18 result = pow(baseUd, exponentUd);
return UD60x18.unwrap(result);

// powWadUp — line 77-80 (IDENTICAL)
UD60x18 result = pow(baseUd, exponentUd);
return UD60x18.unwrap(result);
```

BalancerMath.inGivenOut() calls powWadUp() at line 87 to compute y^r where r = weightOut / weightIn. For non-equal weight pools (e.g., 70/30), rWad is a fractional value that bypasses the integer special cases. The function name promises rounding up — which would protect the pool by requiring slightly more input — but the guarantee is silently dropped. The same issue affects sqrtWadUp()(line 105), which is byte-for-byte identical to sqrtWadDown()(line 98).

Affected Code

- FixedPointMathLib.sol#L49-L81

Impacts
The pool may undercharge input amounts by ~1 wei per swap on non-equal-weight pools. The function names create a false rounding guarantee that callers rely on for pool-protective math.

Remediation

Add a ceiling adjustment to powWadUp() and sqrtWadUp() after the PRBMath call to ensure the result is always rounded up relative to the true value.

Retest

This issue is not fixed.

Bug ID #I006 [Fixed]

Non-Standard ERC20 Approve Pattern in MultiHopRouter May Break For USDT-Style Tokens

Vulnerability Type
Best Practice

Severity
Informational

Description
MultiHopRouter uses IERC20(token).approve(pool, type(uint256).max) in 7 locations, guarded by if (currentAllowance < needed). For tokens like USDT that require the allowance to be zeroed before setting a new non-zero value, this pattern can permanently revert.

The first approve(max) call succeeds (0 → max). However, some non-standard tokens decrement allowance on every transferFrom, even from type(uint256).max. After sufficient transfers, the allowance drops below the required amount, triggering a re-approval. The re-approval call approve(max) reverts because the current allowance is non-zero.

```
// Pattern repeated in 7 locations:
uint256 currentAllowance = IERC20(hop.tokenIn).allowance(address(this), hop.pool);
if (currentAllowance < currentAmount) {
   IERC20(hop.tokenIn).approve(hop.pool, type(uint256).max); // reverts on USDT-style tokens
}
```

Affected Code

- MultiHopRouter.sol#L314

Impacts
Swaps and joins involving USDT-style tokens that require zero-before-set approval could permanently revert after enough transfers decrement the allowance below the required amount.

Remediation
Replace approve with SafeERC20.forceApprove() which handles the zero-before-set pattern.

Retest

This issue is fixed in this

Bug ID #I007 [Acknowledged]

Insufficient SSPCore Upgrade Timelock Delay

Vulnerability Type
Governance

Severity
Informational

Description
WeightedPool implements a two-step timelock for SSPCore upgrades: proposeSSPCoreUpgrade() records a pending address and timestamp, and executeSSPCoreUpgrade() enforces a minimum delay before applying the upgrade. The delay is hardcoded to 2 minutes:

```
uint256 internal constant SSP_UPGRADE_DELAY = 2 minutes;
```

```
function executeSSPCoreUpgrade() external onlyOwner {
   address p = upgradeProposal.pending;
   if (block.timestamp < uint256(upgradeProposal.timestamp) + SSP_UPGRADE_DELAY || p ==
address(0)) {
      revert InvalidConfig();
   }
   ...
}
```

A 2-minute window provides no meaningful opportunity for users or monitoring systems to detect and react to a malicious or erroneous SSPCore replacement. SSPCore controls critical swap settlement logic; a compromised upgrade could redirect fees, manipulate pricing, or drain pool funds. Industry standard timelocks for upgradeable components range from 24 to 48 hours.

Affected Code

- WeightedPool.sol#L27

Impacts
Users and off-chain monitoring systems cannot realistically observe, evaluate, and respond to an SSPCore upgrade proposal within 2 minutes. A compromised owner key could propose and execute a malicious SSPCore replacement before any stakeholder reacts.

Remediation

Increase SSP_UPGRADE_DELAY to a minimum of 24 hours to allow adequate time for review and response.

Retest

Client's comment:- This is for testnet only, for mainnet we will do 48 hours.

Bug ID #G001 [Not Fixed]

Cheaper conditional operators

Vulnerability Type
Gas Optimization (SCWE-082)

Severity
Gas

Description
Upon reviewing the code, it has been observed that the contract uses conditional statements involving comparisons with unsigned integer variables. Specifically, the contract employs the conditional operators x != 0 and x > 0 interchangeably. However, it's important to note that during compilation, x != 0 is generally more cost-effective than x > 0 for unsigned integers within conditional statements.

Affected Code

- PythOracle.sol#L117
- WeightBreakingRewardConfig.sol#L557
- WeightBreakingRewardConfig.sol#L564
- WeightedPoolQueries.sol#L117
- WeightedPoolQueries.sol#L221
- WeightedPoolQueries.sol#L622
- WeightedPoolQueries.sol#L639
- SSPCore.sol#L89
- SSPCore.sol#L96
- WeightedPool.sol#L514
- WeightedPool.sol#L602
- WeightedPoolFactory.sol#L118
- WeightedPoolFactory.sol#L119
- MultiHopRouter.sol#L179
- MultiHopRouter.sol#L218
- MultiHopRouter.sol#L330
- MultiHopRouter.sol#L451

- MultiHopRouter.sol#L492
- MultiHopRouter.sol#L551
- SingleJoinMath.sol#L129
- SingleJoinMath.sol#L132
- PoolOperations.sol#L77
- PoolOperations.sol#L106
- PoolOperations.sol#L121
- PoolOperations.sol#L136
- PoolOperations.sol#L151
- PoolOperations.sol#L185
- PoolOperations.sol#L188
- PoolOperations.sol#L191
- PoolOperations.sol#L224
- OutGivenInMath.sol#L193
- OutGivenInMath.sol#L210
- OutGivenInMath.sol#L238
- OutGivenInMath.sol#L251
- InGivenOutMath.sol#L191
- InGivenOutMath.sol#L208
- InGivenOutMath.sol#L236
- InGivenOutMath.sol#L249
- SwapOperations.sol#L52
- SwapOperations.sol#L100

Impacts

Employing x != 0 in conditional statements can result in reduced gas consumption compared to using x > 0. This optimization contributes to cost-effectiveness in contract interactions.

Remediation

Whenever possible, use the x != 0 conditional operator instead of x > 0 for unsigned integer variables in conditional statements.

Retest

-

Bug ID #G002 [Not Fixed]

Gas Optimization in Increments

Vulnerability Type
Gas optimization (SCWE-082)

Severity
Gas

Description
The contract uses two for loops**,** which use post increments for the variable "**i**".
The contract can save some gas by changing this to **++i**.
++i costs less gas compared to **i++** or **i += 1** for unsigned integers. In **i++**, the compiler has to create a temporary variable to store the initial value. This is not the case with **++i** in which the value is directly incremented and returned, thus, making it a cheaper alternative.

Vulnerable Code

- WeightedPoolQueries.sol#L256
- WeightedPoolQueries.sol#L682
- WeightedPoolQueries.sol#L723
- WeightedPoolFactory.sol#L125
- MultiHopRouter.sol#L298
- MultiHopRouter.sol#L348
- MultiHopRouter.sol#L397
- MultiHopRouter.sol#L378
- MultiHopRouter.sol#L383
- MultiHopRouter.sol#L456
- MultiHopRouter.sol#L497
- MultiHopRouter.sol#L631
- MultiHopRouter.sol#L675
- SingleJoinMath.sol#L99
- PoolOperations.sol#L254
- PoolOperations.sol#L272
- PoolOperations.sol#L308

- PoolOperations.sol#L318
- AdminOperations.sol#L46
- AdminOperations.sol#L60

Impacts
Using **i++** instead of **++i** costs the contract deployment around 600 more gas units.

Remediation
It is recommended to switch to **++i** and change the code accordingly so the function logic remains the same and meanwhile saves some gas.

Retest
-

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

SCS Advocates

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com